# One Whisper Model for Seven Indian Languages

## with Integrated Language Identification

IE 643 Deep Learning — Course Project, Team *blanc*

Complete Explanatory Notes, from First Principles

Shibam Sundar Mahakud • Farhan Al Zayed

Digital audio · log-mel features · WER/CER · Transformers · Whisper · byte-level BPE
Speaker-disjoint splits · balanced sampling · LoRA · vocabulary expansion · CTC
Augmentation · hallucination · bootstrap statistics · calibrated LID · VAD chunking

These notes explain the whole project to a reader who knows undergraduate linear algebra, calculus and probability, and nothing about speech. Every design decision is derived from its mathematics first and then tied to the exact file in `src/` that implements it. Every number is taken from the result files in `reports/` and `runs/`. Worked examples are in green boxes, short self-tests (with solutions) are in purple boxes, and pointers into the code are in grey boxes. A formula sheet and a list of likely viva questions are at the end.

Box legend:

Definition | Theorem / Proposition | Worked example | Check yourself | Intuition | Viva tip | In the code

October 2026

# Contents

**How to Read These Notes & Notation** **5**

**1 The Problem: Multilingual Speech Recognition with Language Identification** **8**
1.1 Two tasks, one model . . . 8
1.2 The seven languages . . . 9
1.3 Goals and scope decisions . . . 9
1.4 The four design principles from the literature . . . 10
1.5 The hardware . . . 10

**2 From Sound to Numbers: Digital Audio and the Log-Mel Spectrogram** **11**
2.1 Sampling . . . 11
2.2 Short-time Fourier transform . . . 12
2.3 The mel scale and the log . . . 12
2.4 The fixed 30-second window . . . 13

**3 Measuring Success: Edit Distance, WER, CER and LID Accuracy** **14**
3.1 Edit distance . . . 14
3.2 WER and CER . . . 15
3.3 Text normalisation . . . 15
3.4 LID metrics . . . 16
3.5 Oracle versus predicted language . . . 16

**4 Sequence-to-Sequence Learning and the Transformer** **18**
4.1 Autoregressive factorisation . . . 18
4.2 Teacher forcing . . . 18
4.3 Attention . . . 19
4.4 The Transformer block . . . 19
4.5 Decoding . . . 20

**5 Whisper: Architecture, Tokenizer, Prompt and Built-in LID** **21**
5.1 What Whisper is . . . 21
5.2 Byte-level BPE and why Indic text is expensive . . . 21
5.3 The prompt and special tokens . . . 22
5.4 Built-in language identification . . . 22
5.5 Baseline numbers that motivated the project . . . 23

**6 Data: Corpora, Pipeline, and Honest Train/Test Splits** **24**

6.1 The corpora . . . 24
6.2 The pipeline . . . 24
6.3 Why the split must be speaker-disjoint . . . 25
6.4 Text leakage . . . 25
6.5 Unique ids . . . 26

**7 Balanced Sampling: Giving Every Language a Fair Share 27**
7.1 Why the mix matters: the objective is a weighted sum . . . 27
7.2 The two-stage algorithm . . . 27
7.3 Evidence: experiment E3 . . . 29

**8 Parameter-Efficient Fine-Tuning with LoRA 30**
8.1 The problem with full fine-tuning . . . 30
8.2 Low-rank adaptation . . . 30
8.3 Where LoRA goes, and the exact parameter count . . . 31
8.4 QLoRA . . . 31
8.5 Experiment E1: LoRA vs QLoRA vs full fine-tuning . . . 32

**9 Teaching Whisper a New Language Token: Odia 33**
9.1 The problem . . . 33
9.2 Weight tying . . . 33
9.3 Initialisation from Bengali . . . 33
9.4 Training only one row: the gradient mask . . . 34
9.5 Choosing the learning rate of the row . . . 34
9.6 A silent trap: `convert_tokens_to_ids` . . . 35
9.7 Result . . . 35

**10 The Encoder LID-CTC Head: Connectionist Temporal Classification 36**
10.1 CTC in general . . . 36
10.2 The one-label special case . . . 37
10.3 Attaching the head to Whisper . . . 37
10.4 From frame posteriors to one decision . . . 38
10.5 Experiment E2: how much LID-CTC? . . . 38

**11 Robustness: Augmentation, Non-Speech Clips and Hallucination 40**
11.1 Why not a denoiser? . . . 40
11.2 The four augmentations . . . 40
11.2.1 Additive noise at a chosen SNR . . . 40
11.2.2 Reverberation . . . 40
11.2.3 Speed perturbation . . . 41
11.2.4 SpecAugment . . . 41
11.3 Hallucination and the non-speech recipe . . . 41

11.4 A metric that could not go below 100% . . . 42
11.5 Experiment E4 . . . 42

**12 The Training Procedure** **44**
12.1 The full loss of one batch . . . 44
12.2 The optimiser . . . 44
12.3 Learning-rate schedule . . . 44
12.4 Gradient clipping and precision . . . 45
12.5 Reproducible randomness and resumable training . . . 45
12.6 Stage 2 in numbers . . . 46

**13 Evaluation Protocol, Baselines and Statistics** **48**
13.1 The test protocol . . . 48
13.2 Baselines . . . 48
13.3 Statistics: how sure are we? . . . 49
13.4 Code-switched speech needs a different metric . . . 49

**14 Results** **51**
14.1 Speech recognition: the full table . . . 51
14.2 The cost of LID errors . . . 52
14.3 Language identification . . . 52
14.4 Robustness . . . 53
14.5 Code-switching with the tolerant metrics . . . 53
14.6 Against Qwen3-ASR on Hindi and English . . . 54
14.7 Summary in the ML-SUPERB style . . . 54

**15 The Indian-English Regression and Two Decode-Time Fixes** **56**
15.1 The symptom . . . 56
15.2 Diagnosis from the per-utterance hypotheses . . . 56
15.3 Fix 1: ASCII-only English decoding . . . 57
15.4 Fix 2: a calibrated LID rule . . . 57
15.5 What would fix it properly . . . 58

**16 The Demo: Long Audio, VAD Chunking and One Language Decision** **59**
16.1 What the demo does . . . 59
16.2 Why the audio must be cut . . . 59
16.3 VAD-based chunking . . . 59
16.4 One language for the whole clip . . . 60
16.5 Measured quality . . . 60

**17 Bugs, Lessons, Limitations and Future Work** **61**
17.1 The bugs that would have changed conclusions . . . 61

17.2 Lessons . . . . . 61
17.3 Limitations . . . . . 62
17.4 Future work . . . . . 62

**18 Formula Sheet and Viva Practice** **63**
18.1 One-page formula sheet . . . . . 63
18.2 Likely viva questions, with answer sketches . . . . . 63

**A Reproduction and Code Map** **66**
A.1 Code map . . . . . 66
A.2 Commands . . . . . 66
A.3 Artefacts of the final model . . . . . 67
A.4 Things to know before re-using results . . . . . 67

# How to Read These Notes & Notation

## The project in one page

The whole project is one continuous story. If you keep this storyline in your head, every chapter has an obvious place.

1. **The task (Ch. 1).** Given a short audio clip of someone speaking Hindi, English, Telugu, Assamese, Odia, Bengali or Marathi (or mixing Hindi/Bengali with English), output (a) the words, in the right script, and (b) which language it was, with a confidence. We want *one* neural network to do both, not a language detector followed by seven recognisers.
2. **Sound as numbers (Ch. 2).** A microphone signal is sampled 16,000 times a second. We cut it into 25 ms frames, take a Fourier transform of each, pool the energies into 128 "mel" bands that follow human pitch perception, and take logarithms. A 30-second clip becomes a $128 \times 3000$ image, the *log-mel spectrogram*.
3. **Scoring (Ch. 3).** We count the minimum number of character insertions, deletions and substitutions that turn the output into the reference (edit distance). Dividing by the reference length gives the character error rate (CER). Language identification (LID) is scored by accuracy and a confusion matrix.
4. **The model family (Ch. 4–5).** Whisper is an encoder–decoder Transformer trained by OpenAI on 680,000 hours of audio. The encoder turns the spectrogram into 1,500 vectors (one per 20 ms). The decoder writes text one token at a time, and its *first* token is the language. Out of the box it is good at English and Hindi but nearly useless at Telugu, Assamese, Bengali and Marathi, and it has no Odia token at all.
5. **Data (Ch. 6–7).** We collect about 700 hours from nine public corpora, make sure no speaker appears in both training and test data, and *rebalance* the mix so that every language gets exactly 90 hours per epoch (plus 70 hours of code-switched speech).
6. **Cheap fine-tuning (Ch. 8).** Instead of updating all 809 million weights, LoRA adds small low-rank correction matrices $\Delta W = \frac{\alpha}{r} BA$ to each layer and trains only those (20.5 million numbers).
7. **A new language token (Ch. 9).** We add `<|or|>` for Odia to the vocabulary, initialise it from Bengali, and make it the only row of the embedding matrix that can change.
8. **A second LID opinion (Ch. 10).** A tiny linear layer on top of the encoder is trained with the CTC loss to shout the language name somewhere in the clip. This gives the encoder a direct LID signal and gives us a second LID prediction.
9. **Robustness (Ch. 11).** Noise, reverberation, speed changes and spectrogram masking at training time; 5% of training clips are pure noise with an *empty* transcript, which teaches the model to stay silent instead of hallucinating.
10. **Training and evaluation (Ch. 12–13).** AdamW, warm-up then linear decay, 11,000 steps of 32 clips, 16.6 hours on one GPU. Evaluation on 23 held-out test sets of 100 utterances, with bootstrap confidence intervals and paired tests.
11. **Results (Ch. 14).** Mean CER falls from 0.696 (vanilla) to 0.113 (true language given) and 0.126 (language predicted by the model). Hallucination on noise falls from about 100% to at

most 1%. We beat the strongest Indic system (IndicConformer) on code-switched speech but not on its own in-domain monolingual data.

12. **The one big failure, and a partial repair (Ch. 15).** Indian-accented English got much worse after fine-tuning. Two decoding-time fixes with no retraining (forbid non-ASCII tokens for English; a two-parameter calibrated LID rule) recover part of it.

13. **Demo, lessons and limits (Ch. 16–17).** A Gradio demo with voice-activity-based chunking, and the bugs that would have changed the conclusions had they not been caught.

## Notation used throughout

| Symbol | Meaning |
|---|---|
| $\mathcal{L}$ | the 7 languages $\{\text{hi}, \text{en}, \text{te}, \text{as}, \text{or}, \text{bn}, \text{mr}\}$ (ISO 639-1 codes) |
| $f_s = 16000$ | sampling rate (samples per second) |
| $x[n]$ | digital waveform, $n = 0, 1, \ldots$ |
| $X[m, k]$ | short-time Fourier transform: frame $m$, frequency bin $k$ |
| $\boldsymbol{F} \in \mathbb{R}^{128\times 3000}$ | log-mel spectrogram of one 30-s window (Whisper large-v3/turbo) |
| $T$ | number of valid encoder frames of a clip, $T = \min(1500, \lfloor 50 \cdot \text{dur} \rfloor + 1)$ |
| $\boldsymbol{h}_1, \ldots, \boldsymbol{h}_{1500} \in \mathbb{R}^d$ | encoder outputs; $d = 1280$ (turbo), 768 (small) |
| $\boldsymbol{y} = (y_1, \ldots, y_U)$ | target token sequence; $\boldsymbol{y}_{<t} = (y_1, \ldots, y_{t-1})$ |
| $V$ | vocabulary size; 51866 for turbo, +1 after adding `<\|or\|>` |
| $\boldsymbol{E} \in \mathbb{R}^{V\times d}$ | token embedding matrix (tied with the output projection) |
| $\boldsymbol{z}_t \in \mathbb{R}^V$ | decoder logits at step $t$; $p_\theta(\cdot \mid \boldsymbol{y}_{<t}, \boldsymbol{x}) = \text{softmax}(\boldsymbol{z}_t)$ |
| $\theta$ | all model parameters |
| $W_0 \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$ | a frozen pretrained weight matrix |
| $A \in \mathbb{R}^{r\times d_{\text{in}}},\ B \in \mathbb{R}^{d_{\text{out}}\times r}$ | LoRA factors; $r = 32$, $\alpha = 64$ |
| $\varnothing$ | CTC blank symbol |
| $\mathcal{B}$ | CTC collapse map (merge repeats, delete blanks) |
| $p_t(c)$ | LID-head posterior of class $c \in \{\varnothing\} \cup \mathcal{L}$ at encoder frame $t$ |
| $\lambda$ | weight of the LID-CTC loss (0.3) |
| $d(\boldsymbol{r}, \boldsymbol{h})$ | Levenshtein (edit) distance between reference $\boldsymbol{r}$ and hypothesis $\boldsymbol{h}$ |
| $S, D, I, N$ | substitutions, deletions, insertions, reference length |
| SNR | signal-to-noise ratio in dB, $10\log_{10}(P_{\text{signal}}/P_{\text{noise}})$ |
| $h_c$ | hours of training audio in corpus $c$; $T_h = 90$ hours per language per epoch |
| $\eta_s$ | learning rate at step $s$ |
| $B_{\text{boot}} = 2000$ | number of bootstrap resamples |
| $b,\ w$ | English bias and head weight of the calibrated LID rule |

**Viva tip**

Three words are used loosely in speech papers, so fix them now. An *utterance* is one recorded sentence or phrase (one row of a manifest). A *corpus* is a dataset (FLEURS, IndicVoices, …). A *test set* in these notes is one (corpus, language) pair, for example `indicvoices/as`; there are 23 of them.

## Where things live

| | |
|---|---|
| `src/` | all code (pipeline, training, evaluation, demo); map in Appendix A |
| `data/processed/` | 16 kHz FLAC audio, JSONL manifests, `splits/train.jsonl`, `dev.jsonl`, `test.jsonl` |
| `runs/s2_turbo/` | final model: `adapter/` (LoRA + Odia row), `lid_head.pt`, `lid_rule.json` |
| `reports/` | every number in these notes: `experiments.md`, `analysis.md`, `test_*.json`, … |
| `docs/PROJECT_LOG.md` | chronological log of decisions and the 26 bugs found |

Chapter 1

# The Problem: Multilingual Speech Recognition with Language Identification

## 1.1 Two tasks, one model

**Definition 1.1: Automatic speech recognition (ASR)**

Let $\mathcal{X}$ be the set of finite audio signals and $\Sigma^*$ the set of finite strings over a character set $\Sigma$ (here: Latin letters plus the Devanagari, Bengali–Assamese, Telugu and Odia scripts). ASR is the task of learning a map $h : \mathcal{X} \to \Sigma^*$ such that $h(\boldsymbol{x})$ is the words spoken in $\boldsymbol{x}$, written in the conventional script of the language.

**Definition 1.2: Spoken language identification (LID)**

Given a finite set of languages $\mathcal{L}$, LID is the task of learning $g : \mathcal{X} \to \mathcal{L}$, or better a probability vector $g(\boldsymbol{x}) \in \Delta^{|\mathcal{L}|-1}$ (the probability simplex), whose largest entry is the language spoken in $\boldsymbol{x}$.

In this project $\mathcal{L}$ is *closed*: the model always answers with one of the seven languages, even for French audio. A closed-set classifier never says "none of these".

**Why not two systems?** The classical pipeline is *LID → route to a language-specific recogniser*. It has two weaknesses:

1. **Error propagation.** If LID is wrong, the wrong recogniser runs and the transcript is garbage. The cost of this is measured by the *oracle-vs-predicted gap*: the CER when the true language is given minus the CER when the predicted language is used (Section 3.5).
2. **No sharing.** Seven separate models learn seven times what speech sounds like. Hindi and Marathi share the Devanagari script, Bengali and Assamese share a script, and all six Indic languages share many sounds and loan words.

A single model with shared representations can transfer knowledge between related languages, and its LID decision is taken by the same network that will transcribe, so the two cannot disagree about what was heard.

## 1.2 The seven languages

| Code | Language | Family | Script | "computer" as usually written |
|---|---|---|---|---|
| hi | Hindi | Indo-Aryan | Devanagari | कंप्यूटर |
| mr | Marathi | Indo-Aryan | Devanagari | संगणक / कॉम्प्युटर |
| bn | Bengali | Indo-Aryan (Eastern) | Bengali | কম্পিউটার |
| as | Assamese | Indo-Aryan (Eastern) | Bengali–Assamese | কম্পিউটাৰ |
| or | Odia | Indo-Aryan (Eastern) | Odia | କମ୍ପ୍ୟୁଟର |
| te | Telugu | Dravidian | Telugu | కంప్యూటర్ |
| en | English | Germanic | Latin | computer |

Three facts in this table drive many later decisions:

- Assamese and Bengali share a script (they differ in a few letters, e.g. Assamese ৰ vs Bengali র), and Hindi and Marathi share Devanagari. LID between these pairs cannot rely on the script of the output; it must come from the audio.
- Odia is closely related to Bengali and Assamese (Eastern Indo-Aryan). This is why the new Odia token is initialised from the Bengali one (Chapter 9).
- Every Indic script is far from ASCII in Unicode. Whisper's tokenizer was built mostly on English text, so Indic text costs many more tokens per second of speech (Chapter 5).

**Code-switching.** In *code-switched* speech the speaker mixes languages inside one sentence: "मेरा computer बंद हो गया". We include Hindi–English and Bengali–English code-switched speech (the MUCS 2021 corpus) for training and testing. A code-switched utterance is labelled with its *matrix* language (hi or bn).

## 1.3 Goals and scope decisions

The goals came from the course proposal and the literature review (`ie643.md`):

1. One Transformer model that transcribes all seven languages and reports the language with a confidence.
2. Keep LID good after fine-tuning (Whisper's LID is known to collapse when it is fine-tuned on new languages).
3. Handle the data imbalance between languages.
4. Be robust to noise and do not invent text on silence or music ("hallucination").
5. Compare honestly against the strongest open baselines: IndicConformer-600M (AI4Bharat), Qwen3-ASR (0.6B, 1.7B) and an external LID model (SpeechBrain ECAPA, VoxLingua107).

Scope decisions made during the project (they override the original slides):

- Final language set: hi, en, te, as, or, bn, mr. Tamil and a low-resource tier (Maithili, Konkani, Santali) were dropped.
- About 90 hours per language instead of the full corpora (disk and compute budget).
- English: LibriSpeech `train-clean-100` was added so that English had comparable hours.

## 1.4 The four design principles from the literature

**Intuition: what the literature review told us to do**

1. **Keep the language token as a training target.** If the decoder is always *told* the language during fine-tuning, it never practises *predicting* it, and LID decays. Making the language token a supervised output keeps it trained.
2. **Add an auxiliary encoder LID loss** (LID-CTC). In ML-SUPERB 2.0 (Interspeech 2025) this gave a 14% relative LID improvement.
3. **Balance the data deterministically** (two-stage upsampling, as in Polyglot-Lion) rather than letting the biggest corpus dominate.
4. **No denoiser in front of the recogniser.** Several 2025–26 studies found that speech enhancement in front of a strong ASR model *hurts*. Instead: training-time augmentation, and non-speech clips with empty transcripts (Canary, Calm-Whisper).

## 1.5 The hardware

Everything ran on one NVIDIA Thor (aarch64, about 112–122 GB of *unified* memory shared by CPU and GPU, CUDA 13). Unified memory has one practical consequence that appears in the log: two Whisper trainings at once exhaust it and both stall, so all trainings were run one after another.

**Viva tip**

If asked "what is new here?", the honest answer is: no single component is new; the contribution is the combination (Whisper turbo + LoRA + a new Odia token + LID-CTC head + balanced sampling + non-speech data) for this specific seven-language set, a careful speaker-disjoint evaluation against strong baselines, and the diagnosis and partial repair of the Indian-English regression.

Chapter 2

# From Sound to Numbers: Digital Audio and the Log-Mel Spectrogram

A neural network needs a tensor. This chapter builds, step by step, the exact tensor Whisper reads.

## 2.1 Sampling

Sound is a pressure wave $x_c(\tau)$, a function of continuous time $\tau$. A microphone and an analogue-to-digital converter record it at equally spaced instants.

**Definition 2.1: Sampled signal**

With sampling rate $f_s$ (samples per second), the digital signal is $x[n] = x_c(n/f_s)$, $n = 0, 1, \ldots, N-1$, stored as floating-point numbers in $[-1, 1]$. A clip of duration $D$ seconds has $N = f_s D$ samples.

**Theorem 2.2: Nyquist–Shannon (statement only)**

If $x_c$ contains no frequency above $f_{\max}$, it is exactly recoverable from its samples whenever $f_s > 2f_{\max}$. Frequencies above $f_s/2$ (the Nyquist frequency) *alias*: they reappear as false lower frequencies.

Whisper uses $f_s = 16{,}000$ Hz, so it sees frequencies up to 8 kHz. That covers what matters for intelligibility (most speech energy is below 4 kHz; fricatives like "s" reach 6–8 kHz). Our corpora come at 16, 22.05, 44.1 or 48 kHz, so every file is *resampled* to 16 kHz once, in `build_manifests.py`.

**In the code**

`build_manifests.py`: `to16k` averages stereo channels to mono and resamples with `scipy.signal.resample_poly(x, 16000/g, sr/g)` where $g = \gcd(\text{sr}, 16000)$. Polyphase resampling upsamples by an integer, low-pass filters (anti-aliasing, so Theorem 2.2 is respected) and downsamples by an integer. Everything is saved as 16 kHz mono FLAC (lossless): 467,515 utterances, 42 GB.

**Check yourself 2.3: Why not just drop every third sample to go from 48 kHz to 16 kHz?**

*Solution.* Dropping samples is resampling without a low-pass filter. Any energy between 8 and 24 kHz in the 48 kHz recording would alias into the 0–8 kHz band and add false components. `resample_poly` filters first. (The demo, in contrast, uses plain linear interpolation `np.interp`, which has no anti-alias filter; this is noted as a known limitation.)

## 2.2 Short-time Fourier transform

Speech is not stationary: the frequencies change every few tens of milliseconds as the mouth moves. So we analyse short overlapping windows.

**Definition 2.4: Short-time Fourier transform (STFT)**

Choose a window length $N_w$, a hop $H$ and a window function $w[n]$ (Whisper: Hann, $w[n] = \sin^2(\pi n/N_w)$). Frame $m$ starts at sample $mH$. Its discrete Fourier transform is

$$X[m,k] = \sum_{n=0}^{N_w-1} x[mH+n]\, w[n]\, e^{-2\pi i k n/N_w}, \qquad k = 0, \dots, N_w/2.$$

The *power spectrogram* is $P[m,k] = |X[m,k]|^2$. Bin $k$ corresponds to frequency $k f_s/N_w$ Hz.

Whisper uses $N_w = 400$ samples (25 ms) and $H = 160$ samples (10 ms), so there are 100 frames per second and 201 frequency bins $k = 0, \dots, 200$, spaced $16000/400 = 40$ Hz apart.

**Intuition: why a window function**

Cutting a frame with a rectangular window creates sharp jumps at the frame edges, and the DFT treats the frame as periodic, so these jumps leak energy into all frequencies ("spectral leakage"). The Hann window goes smoothly to zero at both ends and greatly reduces leakage.

## 2.3 The mel scale and the log

Humans resolve low frequencies finely and high frequencies coarsely. The mel scale models this:

$$\mathrm{mel}(f) = 2595 \log_{10}\!\left(1 + \frac{f}{700}\right).$$

It is roughly linear below 1 kHz and logarithmic above.

**Definition 2.5: Mel filterbank and log-mel spectrogram**

Place $M$ points equally spaced on the mel axis between 0 and mel(8000), convert them back to Hz, and build triangular filters $H_j[k]$ that rise from point $j-1$ to point $j$ and fall to point $j+1$. The mel energies are $\mathcal{M}[j,m] = \sum_k H_j[k]\, P[m,k]$. Whisper then computes

$$L = \log_{10} \max(\mathcal{M}, 10^{-10}), \qquad L \leftarrow \max\big(L,\ \max(L) - 8\big), \qquad \boldsymbol{F} = \frac{L+4}{4}.$$

The three operations after the filterbank have clear purposes:

1. The log turns multiplicative gain (a louder speaker, a different microphone) into an additive offset, and compresses the huge dynamic range of audio power.
2. Clamping at $\max(L) - 8$ keeps only the top 8 orders of magnitude (80 dB) of the clip; quieter values are floored.
3. $(L+4)/4$ roughly centres and scales the values to about $[-1, 1]$, which suits the first convolution layer.

Whisper-small uses $M = 80$ mel bands; large-v3 and large-v3-turbo use $M = 128$.

## 2.4 The fixed 30-second window

Whisper always processes exactly 30 s: shorter audio is padded with zeros, longer audio is cut. So the input is always $\boldsymbol{F} \in \mathbb{R}^{M \times 3000}$ (30 s $\times$ 100 frames/s).

The encoder starts with two 1-D convolutions (kernel 3, GELU); the second has stride 2. So 3000 mel frames become **1500 encoder frames, one per 20 ms, i.e. 50 per second.**

**Proposition 2.6: Number of valid encoder frames**

A clip of $n$ samples ($n/16000$ seconds) occupies $T = \min\big(1500,\ \lfloor 50\, n/16000 \rfloor + 1\big)$ encoder frames; the remaining $1500 - T$ frames see only padding.

*Proof.* Mel frames come every 10 ms, encoder frames every 20 ms, i.e. 50 per second. A clip of $D$ seconds covers encoder frames $0, 1, \ldots, \lfloor 50D \rfloor$, which is $\lfloor 50D \rfloor + 1$ frames, capped at the 1500 available. □

**In the code**

`common.py: enc_frames(n_samples)` implements exactly this formula. It is used everywhere a per-clip "valid length" is needed: the LID-CTC loss (Chapter 10) runs only over valid frames, the LID head sums posteriors only over valid frames, and SpecAugment masks are placed only inside the valid region (mel length $= 2T$).

**Worked example 2.7: Shapes for one 6.4-second Hindi clip**

$n = 6.4 \times 16000 = 102{,}400$ samples. STFT frames: $\approx 640$ real frames, padded to 3000. Mel: $128 \times 3000$. Encoder: $1500 \times 1280$, of which $T = \lfloor 50 \cdot 6.4 \rfloor + 1 = 321$ frames carry speech. The LID head sees 321 valid frames.

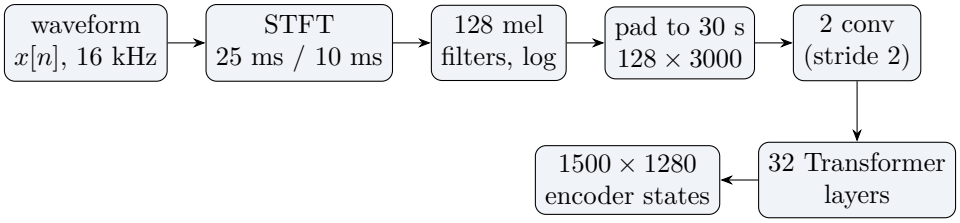



Figure 2.1: Whisper large-v3-turbo front end, from waveform to encoder states.

**Check yourself 2.8: A 27-second Telugu clip**

How many valid encoder frames does it have, and how many *text tokens* might it need if Telugu costs up to 45 Whisper tokens per second?

*Solution.* $T = \lfloor 50 \cdot 27 \rfloor + 1 = 1351$. Text: up to $27 \times 45 = 1215$ tokens, far above the decoder's 448-position limit. Such an utterance cannot be transcribed in one pass; this is why training drops utterances over 440 text tokens and the demo cuts audio into chunks of at most 12 s (Chapters 5 and 16).

Chapter 3

# Measuring Success: Edit Distance, WER, CER and LID Accuracy

Before building anything we must agree on how to score it. A wrong metric would make every later comparison meaningless, and two metric bugs in this project (Chapter 17) would have changed conclusions.

## 3.1 Edit distance

**Definition 3.1: Levenshtein distance**

Let $\boldsymbol{r} = (r_1, \dots, r_n)$ and $\boldsymbol{h} = (h_1, \dots, h_m)$ be sequences over an alphabet. An *edit* is one substitution ($r_i \to h_j$), deletion (drop $r_i$) or insertion (add $h_j$). The Levenshtein distance $d(\boldsymbol{r}, \boldsymbol{h})$ is the minimum number of edits that turns $\boldsymbol{r}$ into $\boldsymbol{h}$.

**Theorem 3.2: Dynamic programme for edit distance**

Let $D[i, j] = d\big((r_1..r_i), (h_1..h_j)\big)$. Then $D[i, 0] = i$, $D[0, j] = j$ and for $i, j \geq 1$

$$D[i, j] = \min\Big\{\, D[i-1, j] + 1, \quad D[i, j-1] + 1, \quad D[i-1, j-1] + \mathbb{1}[r_i \neq h_j] \,\Big\}.$$

$d(\boldsymbol{r}, \boldsymbol{h}) = D[n, m]$, computed in $O(nm)$ time.

*Proof.* An optimal edit script, read left to right, aligns the prefixes. Look at its last operation. Either $r_i$ was deleted (cost 1 plus the best way to turn $r_{1..i-1}$ into $h_{1..j}$), or $h_j$ was inserted (cost 1 plus $D[i, j-1]$), or $r_i$ was aligned with $h_j$ (cost 0 if equal, 1 if substituted, plus $D[i-1, j-1]$). Every script ends in one of these three ways, and each sub-problem is solved optimally by induction on $i + j$, so the minimum of the three is optimal. The base cases need $i$ deletions or $j$ insertions. □

**Worked example 3.3: "kitten" → "sitting"**

| | $\epsilon$ | $s$ | $i$ | $t$ | $t$ | $i$ | $n$ | $g$ |
|---|---|---|---|---|---|---|---|---|
| $\epsilon$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| $k$ | 1 | **1** | 2 | 3 | 4 | 5 | 6 | 7 |
| $i$ | 2 | 2 | **1** | 2 | 3 | 4 | 5 | 6 |
| $t$ | 3 | 3 | 2 | **1** | 2 | 3 | 4 | 5 |
| $t$ | 4 | 4 | 3 | 2 | **1** | 2 | 3 | 4 |
| $e$ | 5 | 5 | 4 | 3 | 2 | **2** | 3 | 4 |
| $n$ | 6 | 6 | 5 | 4 | 3 | 3 | **2** | **3** |

$d = 3$: substitute $k \to s$, substitute $e \to i$, insert $g$. So $S = 2$, $D = 0$, $I = 1$.

## 3.2 WER and CER

**Definition 3.4: Word and character error rates**

For a test set of utterances $i = 1, \ldots, K$ with references $\boldsymbol{r}^{(i)}$ and hypotheses $\boldsymbol{h}^{(i)}$, both *normalised* (Section 3.3),

$$\mathrm{CER} = \frac{\sum_i d_{\mathrm{char}}\left(\boldsymbol{r}^{(i)}, \boldsymbol{h}^{(i)}\right)}{\sum_i |\boldsymbol{r}^{(i)}|_{\mathrm{char}}} = \frac{S + D + I}{N},$$

where sequences are compared character by character (spaces included). WER is the same with words (split on spaces) as symbols.

Note the *micro-average*: total errors over total reference length, not the mean of per-utterance rates. Long utterances weigh more. This is what `jiwer.cer(list, list)` computes and what `metrics.py` reports.

**Proposition 3.5: CER is not bounded by 1**

For one pair, $|m-n| \le d(\boldsymbol{r}, \boldsymbol{h}) \le \max(m, n)$. Hence $\mathrm{CER} \le \max(m, n)/n$, which is unbounded as $m \to \infty$.

*Proof.* Each edit changes the length by at most one, and the length must change by $|m - n|$, so $d \ge |m - n|$. Upper bound: substitute $\min(m, n)$ aligned positions and insert or delete the remaining $|m - n|$ symbols, $\min(m, n) + |m - n| = \max(m, n)$ edits. □

**Intuition: what CER $> 1$ means**

A CER above 1 means the hypothesis is much *longer* than the reference. In this project it almost always means a *repetition loop*: the decoder keeps emitting the same phrase until it hits the 440-token limit. Vanilla Whisper's CER of 2.60 on IndicVoices Assamese is a loop symptom, not a "very precise" measure of quality.

**Why CER is the primary metric here.**

- Indic words are long and inflected (Telugu is agglutinative): one wrong suffix makes the whole word wrong for WER, while CER gives partial credit.
- Spelling variation (e.g. with or without a nukta, two ways of writing a nasal) is common; CER penalises it by one character, WER by a whole word.
- Code-switched references write English words sometimes in Latin, sometimes in Indic script (Section 13.4).

WER is still reported for completeness.

## 3.3 Text normalisation

Two strings that a human would call "the same transcript" can differ in Unicode bytes, punctuation and case. Scoring raw strings would count these as errors. A *single* normaliser is therefore applied to both the training targets and, at scoring time, to both references and hypotheses.

**Definition 3.6: The project's normaliser `normalize(text, lang)`**

1. Unicode NFC (canonical composition).
2. Delete bracketed spans: `<...>`, `[...]`, `(...)` (tags such as `<unintelligible>`).
3. Remove zero-width characters (ZWJ, ZWNJ, ZWSP, BOM).
4. For Indic languages: the IndicNLP normaliser of that language (canonical forms of nukta letters, chandrabindu, etc.).
5. For English: NFKC; keep in-word apostrophes, mapping the curly apostrophe U+2019 to `'`.
6. Lower-case.
7. Replace every Unicode *symbol* (category S*) or *punctuation* (category P*) character by a space. This includes the danda । (U+0964). Combining marks (category M*: matras, viramas) are *kept*.
8. Collapse runs of whitespace; strip.

Digits are *not* converted to words.

**Worked example 3.7: Why NFC matters**

The Devanagari letter क़ (qa) can be stored as one code point U+0958 or as two, क U+0915 followed by the nukta U+093C. They look identical, but a character-level comparison would count one substitution plus one insertion. NFC (plus IndicNLP's nukta handling) maps both to the same sequence.

**Viva tip: two known side effects of the normaliser**

(1) Digits are not verbalised, so "25" vs "twenty five" counts as errors; this hurts Svarah (Chapter 15). (2) Parenthesised text is deleted. For IndicVoices this removes tags, but in FLEURS the parenthesised words are usually *read aloud*, so 5–8% of FLEURS rows lose spoken words from both targets and references. Re-scoring shows this inflates FLEURS CER by 0.005–0.02 for *every* system (e.g. Stage 2 English 0.042 → 0.028 if kept). Rankings barely change. This was found in the fourth code review and left open because fixing it changes the training targets.

## 3.4 LID metrics

**Definition 3.8: LID accuracy and confusion matrix**

For a test set whose true language is $\ell$, LID accuracy is the fraction of utterances with $\hat{\ell} = \ell$. Pooling all test sets, the confusion matrix $C \in \mathbb{N}^{7\times 7}$ has $C_{\ell\ell'} = \#\{\text{utterances of language } \ell \text{ predicted as } \ell'\}$. Recall of $\ell$ is $C_{\ell\ell}/\sum_{\ell'} C_{\ell\ell'}$. *Macro* accuracy averages per-set accuracies with equal weight.

## 3.5 Oracle versus predicted language

Whisper must be given a language token before it transcribes. We decode every test utterance twice:

- **Oracle LID**: the true language token is forced. This measures pure recognition quality.
- **Predicted LID**: the model's own LID decision is used. This is what a user experiences.

The difference $\mathrm{CER}_{\mathrm{pred}} - \mathrm{CER}_{\mathrm{oracle}}$ is the *routing cost* of LID errors. The literature reports about 4 CER points for a strong LID-then-ASR system (TalTech, ML-SUPERB 2.0); we measure ours in Chapter 14.

**In the code**

`eval_whisper.py` decodes with the oracle token, predicts the language, and *re-decodes only the utterances whose predicted language differs* (the same token gives the same greedy decode, so the oracle output is reused). It reports `oracle_norm`, `oracle_raw`, `pred_norm`, `lid_acc`, `lid_conf`, and the LID-head accuracy.

**Check yourself 3.9: Micro vs macro**

Set A has 2 utterances: reference lengths 10 and 90 characters, with 5 and 9 errors. Compute the micro CER and the mean of per-utterance CERs.
*Solution.* Micro: $(5+9)/(10+90) = 0.14$. Mean of rates: $(0.5+0.1)/2 = 0.30$. The short utterance dominates the macro value. The project uses micro CER within a test set, and reports the plain mean over test sets as the summary ("mean over 23 sets").

Chapter 4

# Sequence-to-Sequence Learning and the Transformer

ASR maps a sequence (audio frames) to another sequence (text tokens) of a different, unknown length. This chapter derives the standard tool for that, the encoder–decoder Transformer, at the level needed to understand every later chapter.

## 4.1 Autoregressive factorisation

Let $\boldsymbol{y} = (y_1, \dots, y_U)$ be the target tokens, ending with an end-of-text token `<|endoftext|>` (EOT). By the chain rule of probability, *any* distribution over sequences can be written as

$$p_\theta(\boldsymbol{y} \mid \boldsymbol{x}) = \prod_{t=1}^{U} p_\theta(y_t \mid \boldsymbol{y}_{<t}, \boldsymbol{x}).$$

So it suffices to model one next-token distribution at a time. The decoder outputs logits $\boldsymbol{z}_t \in \mathbb{R}^V$ and $p_\theta(\cdot \mid \boldsymbol{y}_{<t}, \boldsymbol{x}) = \mathrm{softmax}(\boldsymbol{z}_t)$, where $\mathrm{softmax}(\boldsymbol{z})_v = e^{z_v} / \sum_u e^{z_u}$.

**Definition 4.1: Maximum likelihood and token cross-entropy**

Given training pairs $(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})$, we minimise the negative log-likelihood, averaged over target tokens:

$$\mathcal{L}_{\mathrm{CE}}(\theta) = \frac{1}{\sum_i U_i} \sum_i \sum_{t=1}^{U_i} -\log p_\theta\big(y_t^{(i)} \mid \boldsymbol{y}_{<t}^{(i)}, \boldsymbol{x}^{(i)}\big).$$

**Proposition 4.2: Gradient of softmax cross-entropy**

For one position with logits $\boldsymbol{z}$ and target $y$, $\ell(\boldsymbol{z}) = -\log \mathrm{softmax}(\boldsymbol{z})_y$ has gradient $\nabla_{\boldsymbol{z}} \ell = \boldsymbol{p} - \boldsymbol{e}_y$, where $\boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z})$ and $\boldsymbol{e}_y$ is the one-hot vector of $y$.

*Proof.* $\ell = -z_y + \log \sum_u e^{z_u}$. Then $\partial \ell / \partial z_v = -\mathbb{1}[v = y] + e^{z_v} / \sum_u e^{z_u} = p_v - \mathbb{1}[v = y]$. □

**Intuition**

The gradient pushes the correct logit up by $1 - p_y$ and every wrong logit down by its own probability. Every token in the vocabulary receives a gradient at every position. This matters in Chapter 9: the new Odia token is pushed down at every position of every non-Odia utterance, and up only where it is the target.

## 4.2 Teacher forcing

During training the decoder is fed the *true* previous tokens, not its own predictions ("teacher forcing"). In practice the decoder input is the target sequence shifted right by one, with a start token in front:

$$\text{decoder input} = (\texttt{SOT}, y_1, \dots, y_{U-1}), \qquad \text{labels} = (y_1, \dots, y_U).$$

All $U$ positions are then computed in one parallel forward pass (with a causal mask, below). Positions that should not be trained get the label $-100$, which PyTorch's cross-entropy ignores.

## 4.3 Attention

**Definition 4.3: Scaled dot-product attention**

Given queries $Q \in \mathbb{R}^{n\times d_k}$, keys $K \in \mathbb{R}^{m\times d_k}$ and values $V \in \mathbb{R}^{m\times d_v}$,

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}} + M\Big)V,$$

with the softmax taken row by row and $M$ a mask ($0$ = allowed, $-\infty$ = forbidden). Row $i$ of the output is a convex combination of the value vectors, weighted by how well query $i$ matches each key.

**Proposition 4.4: Why divide by $\sqrt{d_k}$**

If the entries of $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^{d_k}$ are independent with mean 0 and variance 1, then $\mathbb{E}[\boldsymbol{q}^\top \boldsymbol{k}] = 0$ and $\mathrm{Var}(\boldsymbol{q}^\top \boldsymbol{k}) = d_k$. Dividing by $\sqrt{d_k}$ restores unit variance.

*Proof.* $\boldsymbol{q}^\top \boldsymbol{k} = \sum_j q_j k_j$. Each term has mean $\mathbb{E}q_j \mathbb{E}k_j = 0$ and variance $\mathbb{E}[q_j^2]\mathbb{E}[k_j^2] = 1$, and the terms are independent, so variances add to $d_k$. □

Without the scaling, large dot products saturate the softmax (almost one-hot), whose gradients are then nearly zero.

**Multi-head attention.** The model dimension $d$ is split into $H$ heads of size $d/H$. Each head has its own projections $W_h^Q, W_h^K, W_h^V \in \mathbb{R}^{d\times d/H}$; the head outputs are concatenated and multiplied by $W^O \in \mathbb{R}^{d\times d}$. In HuggingFace's Whisper these four matrices are the modules `q_proj`, `k_proj`, `v_proj`, `out_proj`: exactly the names LoRA targets in Chapter 8.

**Three kinds of attention in an encoder–decoder.**

1. **Encoder self-attention**: queries, keys and values all come from the encoder; no mask (every frame sees every frame).
2. **Decoder causal self-attention**: from the decoder; $M_{ij} = -\infty$ for $j > i$, so position $i$ cannot look at future tokens. This is what makes parallel teacher-forced training equivalent to left-to-right generation.
3. **Cross-attention**: queries from the decoder, keys and values from the encoder output. This is how the text "reads" the audio.

## 4.4 The Transformer block

Whisper uses *pre-norm* residual blocks. For an encoder layer with input $H \in \mathbb{R}^{n\times d}$:

$$H \leftarrow H + \mathrm{MHA}(\mathrm{LN}(H)), \qquad H \leftarrow H + \mathrm{MLP}(\mathrm{LN}(H)), \qquad \mathrm{MLP}(\boldsymbol{u}) = W_2\,\mathrm{GELU}(W_1\boldsymbol{u} + \boldsymbol{b}_1) + \boldsymbol{b}_2,$$

with $W_1 \in \mathbb{R}^{4d\times d}$ (`fc1`) and $W_2 \in \mathbb{R}^{d\times 4d}$ (`fc2`). A decoder layer has three sub-blocks: causal self-attention, cross-attention, MLP. Positional information is added at the input (sinusoidal for the encoder, learned for the decoder), because attention itself is permutation-invariant.

**Intuition: residual streams**

Each block *adds* a correction to a running "residual stream". If a block's output is zero, the layer is the identity. This is exactly why LoRA can be initialised so that the fine-tuned model starts identical to the pretrained one: an added correction of zero changes nothing (Chapter 8).

## 4.5 Decoding

At inference time there is no reference, so tokens are generated one by one. This project uses **greedy decoding**: $\hat{y}_t = \arg\max_v p_\theta(v \mid \hat{\boldsymbol{y}}_{<t}, \boldsymbol{x})$ until EOT or a maximum of 440 new tokens.

**KV cache.** At step $t$ the keys and values of positions $1..t-1$ do not change, so they are cached; each step computes only the new position. Generating $U$ tokens costs $O(U)$ decoder passes of one position each instead of $O(U^2)$ total positions. The encoder runs once per clip.

**Check yourself 4.5: Why is training parallel but inference sequential?**

*Solution.* In training all inputs $(\texttt{SOT}, y_1, \ldots, y_{U-1})$ are known in advance (teacher forcing), and the causal mask guarantees position $t$ uses only positions $< t$, so all positions are computed at once. In inference $\hat{y}_t$ is needed to compute position $t+1$, so steps must run one after another.

**Viva tip**

A classic question: "what is exposure bias?" With teacher forcing the decoder never sees its own mistakes during training; at test time one wrong token can push it into states it never saw. Repetition loops are one visible symptom. The hard 440-token cap and the "loop rate" metric (fraction of outputs that hit the cap) are the project's guards.

Chapter 5

# Whisper: Architecture, Tokenizer, Prompt and Built-in LID

## 5.1 What Whisper is

Whisper (Radford et al., 2023) is an encoder–decoder Transformer (Chapter 4) trained by OpenAI on about 680,000 hours of weakly labelled web audio in about 100 languages. It does transcription, translation to English, timestamps and language identification, all through the decoder's token stream.

| | Whisper-small | large-v3 | **large-v3-turbo** (our base) |
|---|---|---|---|
| Parameters | 244 M | 1.55 B | 809 M |
| Encoder layers | 12 | 32 | 32 |
| Decoder layers | 12 | 32 | **4** |
| Width $d$ / heads | 768 / 12 | 1280 / 20 | 1280 / 20 |
| MLP width | 3072 | 5120 | 5120 |
| Mel bins $M$ | 80 | 128 | 128 |
| Vocabulary $V$ | 51,865 | 51,866 | 51,866 |
| Max decoder positions | 448 | 448 | 448 |

*Turbo* is large-v3 with the decoder pruned from 32 to 4 layers and then fine-tuned further. Generation runs one decoder pass per token, so a 4-layer decoder makes decoding about 6 times faster, while the large 32-layer encoder keeps most of the acoustic quality. Whisper-small was the development model for experiments E1–E4 (Chapter 14); turbo is the final model.

## 5.2 Byte-level BPE and why Indic text is expensive

**Definition 5.1: Byte-level byte-pair encoding (BPE)**

Text is first written as UTF-8 bytes (256 base symbols). BPE training repeatedly finds the most frequent adjacent pair of symbols in a training corpus and merges it into a new symbol, until the vocabulary has the desired size. Tokenising a new string applies the learned merges greedily. Any string can be tokenised (in the worst case one token per byte).

UTF-8 encodes an ASCII letter in 1 byte and a character of an Indic script in 3 bytes. Whisper's merges were learned mostly from English-heavy text, so frequent English words become single tokens, while Indic text is split into short pieces, often single characters or bytes.

**Worked example 5.2: Token cost measured on our data**

Measured with `prep_tokens.py` on the normalised transcripts: Indic speech costs **17–45 Whisper tokens per second**, English about 3. Of Telugu, Assamese and Bengali reference transcripts, 41–58% exceed 200 tokens, and some exceed 448, the decoder's hard limit.

**Proposition 5.3: Token budget**

The decoder sequence has at most 448 positions. Our prompt uses 4 (Section 5.3) and the target ends with EOT. So a transcript can be learned in one pass only if it has at most $448 - 4 - 1 = 443$ text tokens. The project uses 440, leaving a small margin.

**In the code**

`common.py: MAX_TEXT_TOKENS = 440`. Training drops utterances with more than 440 text tokens (8,131 utterances, 2.1% of the training split). Test keeps everything; the 550 over-budget test utterances simply count as errors (they cannot be completed). All decoding uses `max_new_tokens=440`. The first baseline run used 200, which truncated Indic outputs (bug 5).

## 5.3 The prompt and special tokens

Whisper's decoder is conditioned by special tokens. For plain transcription without timestamps the sequence is

$$\underbrace{\texttt{SOT}}_{\text{start}}\ \underbrace{\texttt{<|hi|>}}_{\text{language}}\ \texttt{<|transcribe|>}\ \texttt{<|notimestamps|>}\ \text{text tokens}\ \ldots\ \texttt{EOT}.$$

**Definition 5.4: The project's training sequence**

For an utterance of language $\ell$ with text tokens $w_1..w_K$:

$$\begin{aligned}\text{decoder input} &= [\texttt{SOT},\ \texttt{<|}\ell\texttt{|>},\ \texttt{<|transcribe|>},\ \texttt{<|notimestamps|>},\ w_1, \ldots, w_K]\\ \text{labels} &= [\texttt{<|}\ell\texttt{|>},\ \texttt{<|transcribe|>},\ \texttt{<|notimestamps|>},\ w_1, \ldots, w_K,\ \texttt{EOT}]\end{aligned}$$

The first label is the language token: **the language is a supervised target**, predicted from `SOT` and the audio alone.

The usual HuggingFace recipe puts the language in the *forced* prefix, so the model is told the language and never learns to predict it. That is the documented cause of LID collapse after fine-tuning. Making it a label keeps LID trained.

## 5.4 Built-in language identification

**Definition 5.5: Closed-set decoder-token LID**

Feed only [`SOT`] to the decoder. Let $\boldsymbol{z}_1$ be the logits of the first predicted position. Restrict to the language tokens of $\mathcal{L}$:

$$\hat{p}_{\text{tok}}(\ell \mid \boldsymbol{x}) = \frac{\exp z_{1,\text{id}(\ell)}}{\sum_{\ell' \in \mathcal{L}} \exp z_{1,\text{id}(\ell')}}, \qquad \hat{\ell} = \arg\max_{\ell \in \mathcal{L}} \hat{p}_{\text{tok}}(\ell \mid \boldsymbol{x}).$$

This is a closed-set softmax over 7 of the ≈51,867 logits. Vanilla Whisper's version of this collapses outside English and Hindi on our test data: Telugu 14%, Assamese 0%, Bengali 51%, Marathi 52% (Chapter 14).

**Viva tip: Odia**

Whisper's 100 languages do *not* include Odia. This was checked three ways: the official `whisper/tokenizer.py`, the HF `generation_config.lang_to_id` (100 entries, no `or` or `od`), and the vocabulary itself. So for vanilla Whisper, Odia cannot be predicted or forced at all, and Odia CER is above 1 (it writes Odia speech in other scripts or loops).

## 5.5 Baseline numbers that motivated the project

Zero-shot test CER (100 utterances per set, mean over non-code-switched corpora of each language; `reports/baselines/summary.md`):

| Language | Whisper-small | Whisper-turbo | IndicConformer-600M | Turbo LID accuracy |
|---|---|---|---|---|
| English | 0.04 | 0.03 | n/a | 100% |
| Hindi | 1.09 | 0.21 | 0.05 | 81% |
| Marathi | 1.13 | 0.40 | 0.05 | 52% |
| Bengali | 1.68 | 0.72 | 0.05 | 51% |
| Telugu | 2.11 | 0.66 | 0.07 | 14% |
| Assamese | 1.58 | 1.90 | 0.06 | 0% |
| Odia | 1.66 | 1.46 | 0.07 | no token |

Vanilla Whisper is usable only for English and Hindi. IndicConformer (an Indic-specialised Conformer trained on AI4Bharat data) is far better on Indic CER but has no English and no LID. The project's aim is to close as much of this gap as possible *inside one model that also does English, code-switching and LID.*

**Check yourself 5.6: Why does a 4-layer decoder hardly hurt accuracy?**

*Solution.* In ASR most of the work is acoustic: deciding which sounds were spoken. That is done by the encoder, which is kept at 32 layers. The decoder mainly acts as a conditional language model that reads the encoder through cross-attention; a shallow decoder is enough for that. Since generation cost is proportional to decoder depth times number of tokens, cutting the decoder gives the speed-up for little accuracy loss.

Chapter 6

# Data: Corpora, Pipeline, and Honest Train/Test Splits

## 6.1 The corpora

| Corpus | Languages | Character | Role |
|---|---|---|---|
| IndicVoices (AI4Bharat) | as, bn, hi, mr, or, te | spontaneous, many speakers, the hardest | train / dev / test |
| Kathbath (AI4Bharat) | bn, hi, mr, or, te (no as) | read sentences, phone recordings | train / dev / test |
| Shrutilipi | as, bn, hi, mr, or, te | All India Radio news, read | train only |
| FLEURS (Google) | all 7 | read Wikipedia sentences | train / dev / test |
| LibriSpeech | en | read audiobooks (clean-100, dev/test-clean) | train / dev / test |
| MUCS 2021 | hi–en, bn–en | code-switched lectures | train / dev / test |
| Svarah | en (Indian accent) | 117 speakers, 65 districts | test only |
| Lahaja | hi (many accents) | multi-accent Hindi | test only |
| MUSAN (noise, music), RIRs | – | augmentation and robustness tests | 80% train / 20% eval |

Training hours per language after splitting (from `reports/splits.txt`):

| | FLEURS | IndicVoices | Kathbath | Shrutilipi | LibriSpeech | MUCS (code-switch) |
|---|---|---|---|---|---|---|
| hi | 6.6 | 55.5 | 14.1 | 21.0 | – | 77.6 |
| mr | 11.8 | 48.2 | 13.6 | 22.2 | – | – |
| bn | 10.6 | 44.6 | 13.3 | 21.0 | – | 40.0 |
| as | 10.4 | 54.0 | – | 23.9 | – | – |
| or | 3.4 | 54.7 | 13.4 | 24.7 | – | – |
| te | 7.9 | 49.4 | 14.6 | 21.9 | – | – |
| en | 7.4 | – | – | – | 100.6 | – |

These are the *downloaded subsets* (some shards of each corpus), chosen so that every language has roughly 85–110 hours.

## 6.2 The pipeline

1. `audit_data.py`: reads only metadata and writes hours, utterances and speakers per corpus, language and split to `reports/data_audit.csv`.
2. `build_manifests.py`: decodes every corpus to 16 kHz mono FLAC and writes one JSON line per utterance: `id, corpus, lang, split, path, dur, text, text_norm, speaker,`

`cs`. It drops clips longer than 30 s (they do not fit Whisper's window), shorter than 0.3 s, or with an empty normalised text. MUCS long recordings are cut using its `segments` file. Result: 467,515 utterances.

3. `make_splits.py`: assigns each utterance to train, dev or test (next section).
4. `prep_tokens.py`: adds `ntok`, the Whisper token count of `text_norm`.
5. `balanced_mix.py`: builds the 700-hour training mix (Chapter 7).

## 6.3 Why the split must be speaker-disjoint

**Definition 6.1: Generalisation error**

Let $\mathcal{D}$ be the distribution of (audio, transcript) pairs a deployed system will meet, and $\ell$ a loss. The quantity we want is $R(h) = \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim\mathcal{D}}[\ell(h(\boldsymbol{x}), \boldsymbol{y})]$. A test set estimates $R(h)$ by an average $\hat{R}(h) = \frac{1}{K}\sum_i \ell(h(\boldsymbol{x}_i), \boldsymbol{y}_i)$, which is unbiased only if the test pairs are drawn from $\mathcal{D}$ *independently of the training data.*

A real user is (almost always) a *new speaker*. If test speakers also appear in training, the model may have memorised their voice, accent and microphone, and $\hat{R}$ underestimates $R$. The test pairs are then not independent of the training sample.

**Worked example 6.2: The official splits were not speaker-disjoint**

`make_splits.py` checks the official IndicVoices and Kathbath `train` vs `valid` splits:

| Corpus | train speakers | valid speakers | shared |
|---|---|---|---|
| IndicVoices | 4,915 | 2,702 | **1,217** |
| Kathbath | 170 | 91 | **40** |

Fix: every `valid` utterance whose speaker occurs in `train` is moved to train. Dev and test are drawn only from valid-only speakers (30% of those speakers to dev, the rest to test). The script then re-checks: train∩dev = train∩test = dev∩test = ∅ for IndicVoices, Kathbath and MUCS.

**Role assignment per corpus.**

- IndicVoices, Kathbath: as above.
- Shrutilipi: train only (no speaker ids exist, so a speaker-disjoint test split is impossible).
- FLEURS, LibriSpeech: their official train/dev/test (already speaker-disjoint by construction).
- MUCS: official test is test; 5% of training *recordings* go to dev (recording ids act as speakers).
- Svarah, Lahaja: test only.

## 6.4 Text leakage

Many corpora reuse sentences: different speakers read the same prompt. If a test sentence's text is in training, the decoder may recall it as a language model rather than recognise it.

**Definition 6.3: Leakage removal**

For each language $\ell$, let $\mathcal{T}_\ell$ be the set of normalised texts of all dev and test utterances of $\ell$. Every training utterance of $\ell$ whose normalised text is in $\mathcal{T}_\ell$ is removed (`role = dropped_leak`).

Removed: IndicVoices hi 5,005, mr 2,862, or 2,667, te 2,042, bn 2,036, as 1,357; MUCS hi 4,964, bn 2,538; a handful in LibriSpeech and Shrutilipi. (MUCS lectures repeat phrases often, hence the large count.)

Final split: **384,027 train** / **14,883 dev** / **45,104 test** utterances.

## 6.5 Unique ids

Bug 8: 19,361 ids were not unique (FLEURS sentence ids are shared by several speakers; some shards had identical file names in every language). Paths were always unique, so no audio was lost, but ids are used to pair hypotheses across systems in the statistics of Chapter 13, so they were made unique and the splits regenerated.

**Check yourself 6.4: Why is Shrutilipi train-only, and why does that not bias the results?**

*Solution.* It has no speaker labels, so we cannot guarantee that a Shrutilipi test speaker is unseen. Using it only for training is safe: the speakers of the other corpora's test sets are a different population (AI4Bharat recruits vs radio newsreaders), and none of our test sets is drawn from Shrutilipi.

**Viva tip**

If an examiner asks "how do you know your numbers are not inflated?", the three checks are: speaker-disjoint dev/test (re-verified with 0 overlap), text-leakage removal, and test sets never used for any decision (E1–E4 used the dev split only; the test split was used once, for the final model and baselines). The one exception is calibration of the LID rule on Svarah, which used a *speaker-disjoint* Svarah subset (Chapter 15).

Chapter 7

# Balanced Sampling: Giving Every Language a Fair Share

## 7.1 Why the mix matters: the objective is a weighted sum

Let the training mix sample language $\ell$ with probability $\pi_\ell$, and let $R_\ell(\theta)$ be the expected per-token loss on language $\ell$. The expected mini-batch loss that SGD minimises is

$$\mathbb{E}[\mathcal{L}_{\text{batch}}(\theta)] \approx \sum_{\ell \in \mathcal{L}} \pi_\ell \, R_\ell(\theta), \qquad \mathbb{E}[\nabla \mathcal{L}_{\text{batch}}] \approx \sum_{\ell} \pi_\ell \nabla R_\ell(\theta).$$

("≈" because the loss is averaged over tokens, and languages differ in tokens per utterance.)

**Proposition 7.1: The mix chooses the objective**

Training on a mix with weights $\pi$ minimises the $\pi$-weighted risk $\sum_\ell \pi_\ell R_\ell$. If we report results as a plain average over languages, the matching training objective is $\pi_\ell = 1/|\mathcal{L}|$.

*Proof.* Immediate from linearity of expectation over the sampling of the language of each example. □

With natural proportions ($\pi_\ell \propto$ hours available), a language with 5% of the data gets 5% of the gradient. Its errors barely move the objective, so the model under-fits it. The price of equalising is repetition: a small corpus is seen several times per epoch, which risks over-fitting. The design below caps that repetition.

## 7.2 The two-stage algorithm

**Definition 7.2: Two-stage deterministic balanced upsampling (Polyglot-Lion style)**

Constants: $T_h = 90$ hours per language, replication cap $\rho = 3$, share cap $\kappa = 0.7$, code-switch fraction $f = 0.1$.

**Stage 1 (within a language).** Let $h_c$ be the hours of corpus $c$ for this language and $h_{\max} = \max_c h_c$. Set

$$s_c = h_c \cdot \min\Big(\rho, \; \frac{h_{\max}}{h_c}\Big), \qquad \text{share}_c = \frac{s_c}{\sum_{c'} s_{c'}}.$$

If some shares exceed $\kappa$, set those to $\kappa$ and rescale the others to fill $1 - \kappa \cdot \#\text{capped}$ (repeat up to 10 times; skipped if every corpus is over the cap).

**Stage 2 (across languages).** Corpus $c$ contributes exactly $T_h \cdot \text{share}_c$ hours: $\lfloor \text{target}/h_c \rfloor$ full copies plus a seeded random subset for the remainder (fewer than one copy means sub-sampling).

**Code-switch pool.** MUCS gets $\frac{f}{1-f} \cdot |\mathcal{L}| \, T_h$ hours, split 60% Hindi–English, 40% Bengali–English.

Stage 1 says: *inside* a language, small corpora are pulled up toward the largest, but never replicated more than 3 times, and no single corpus may take more than 70% of the language (so the language keeps several domains: read, spontaneous, news).

**Proposition 7.3: Total size of the mix**

The mix has $|\mathcal{L}|T_h/(1-f)$ hours, of which a fraction $f$ is code-switched. With 7 languages, $T_h = 90$, $f = 0.1$: $630 + 70 = \mathbf{700}$ hours.

*Proof.* Monolingual: $|\mathcal{L}|T_h$. Code-switch: $\frac{f}{1-f}|\mathcal{L}|T_h$. Sum: $|\mathcal{L}|T_h\big(1+\frac{f}{1-f}\big) = \frac{|\mathcal{L}|T_h}{1-f}$. Its code-switch fraction is $\frac{f}{1-f}\big/\frac{1}{1-f} = f$. □

**Worked example 7.4: Hindi**

$h$: FLEURS 6.6, IndicVoices 55.5, Kathbath 14.1, Shrutilipi 21.0; $h_{\max} = 55.5$.

| Corpus | $h_c$ | $h_{\max}/h_c$ | $s_c$ | share | target (h) | replication |
|---|---|---|---|---|---|---|
| FLEURS | 6.6 | 8.41 | 19.8 | 0.114 | 10.3 | 1.56× |
| IndicVoices | 55.5 | 1.00 | 55.5 | 0.321 | 28.9 | 0.52× |
| Kathbath | 14.1 | 3.94 | 42.3 | 0.244 | 22.0 | 1.56× |
| Shrutilipi | 21.0 | 2.64 | 55.5 | 0.321 | 28.9 | 1.37× |
| Sum | 97.2 | | 173.1 | 1.000 | 90.0 | |

No share exceeds 0.7. IndicVoices is *sub-sampled* to about half per epoch, while FLEURS and Kathbath are repeated. (The real run uses hours after the 440-token filter, so its numbers differ slightly.)

**Worked example 7.5: English: the share cap in action**

$h$: FLEURS 7.4, LibriSpeech 100.6. Stage 1: $s = (3 \cdot 7.4,\ 100.6) = (22.2,\ 100.6)$, shares $(0.181,\ 0.819)$. LibriSpeech exceeds $\kappa = 0.7$, so it is capped at 0.7 and FLEURS gets 0.3. Targets: FLEURS 27 h, i.e. **3.65**× replication (the share cap overrides the replication cap; the log notes "English FLEURS replicated 3.6×"), LibriSpeech 63 h.

**Check yourself 7.6: Odia FLEURS**

Odia: FLEURS 3.4, IndicVoices 54.7, Kathbath 13.4, Shrutilipi 24.7 hours. Find FLEURS's target hours.

*Solution.* $h_{\max} = 54.7$. $s$: FLEURS $3 \cdot 3.4 = 10.2$; IndicVoices 54.7; Kathbath $3 \cdot 13.4 = 40.2$; Shrutilipi $24.7 \cdot \min(3, 2.21) = 54.7$. Sum 159.8. FLEURS share $10.2/159.8 = 0.064$, target 5.7 h (1.69×).

**The result.** One deterministic file, `splits/train_mix_epoch0.jsonl`: 349,592 utterance references (no audio is copied), exactly 90 h per language plus 70 h of code-switch (42 h hi–en, 28 h bn–en). Language tags are kept, because they are needed as the <|ℓ|> target and the LID-CTC target. (Polyglot-Lion drops tags so the model learns LID implicitly; we need an explicit LID output, so we keep them.)

**Intuition: deterministic vs temperature sampling**

The classical alternative samples language $\ell$ with $\pi_\ell \propto n_\ell^{1/\tau}$ (temperature $\tau$). It works but $\tau$ is a sensitive knob and the realised hours per language are random. The two-stage method gives exactly equal hours by construction.

## 7.3 Evidence: experiment E3

Whisper-small, LoRA, Hindi/Telugu/Assamese with a deliberately skewed pool (hi 97 h, te 26 h, as 5 h), same number of steps. Dev CER:

| Set | unbalanced | balanced | change |
|---|---|---|---|
| FLEURS as | 0.242 | **0.182** | −0.060 |
| IndicVoices as | 0.342 | **0.278** | −0.064 |
| FLEURS te | 0.129 | 0.119 | −0.010 |
| FLEURS hi | **0.096** | 0.111 | +0.015 |
| IndicVoices hi | **0.097** | 0.113 | +0.016 |

Exactly what Proposition 7.1 predicts: the scarcest language (Assamese) gains a lot, the richest (Hindi) loses a little. Decision: balance.

**In the code**

`balanced_mix.py`: `corpus_shares` (Stage 1 with the cap loop), `take` (full copies + seeded remainder), `build` (Stage 2 + code-switch). Run `python balanced_mix.py 1` to get a reshuffled second epoch. Bug 23: a language with a single corpus would have been capped at 70% of $T_h$; fixed by skipping the cap when every corpus is over it.

**Viva tip**

Note what one epoch of this mix means for IndicVoices: it is sub-sampled to 0.5–0.7× per epoch. Stage 2 trained exactly one epoch of mix 0, so part of the IndicVoices training data was never seen. A second epoch should use a reseeded mix (seed 1), not mix 0 again.

Chapter 8

# Parameter-Efficient Fine-Tuning with LoRA

## 8.1 The problem with full fine-tuning

Full fine-tuning updates all 809 M parameters of turbo. AdamW stores two extra numbers per parameter (first and second moments), plus gradients, so memory is several times the model size. It also lets the model drift far from what it learned on 680,000 hours, which risks *catastrophic forgetting* and over-fitting a few hundred hours.

## 8.2 Low-rank adaptation

**Definition 8.1: LoRA (Hu et al., 2021)**

Take a frozen linear layer $\boldsymbol{u} \mapsto W_0\boldsymbol{u}$ with $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. LoRA replaces it by

$$\boldsymbol{u} \;\mapsto\; W_0\boldsymbol{u} + \frac{\alpha}{r}\, BA\boldsymbol{u}, \qquad A \in \mathbb{R}^{r \times d_{\text{in}}},\; B \in \mathbb{R}^{d_{\text{out}} \times r},\; r \ll \min(d_{\text{in}}, d_{\text{out}}).$$

Only $A$ and $B$ are trained. Initialisation: $A$ random (Kaiming-uniform), $B = 0$. Dropout (here 0.05) is applied to $\boldsymbol{u}$ on the LoRA branch only.

**Proposition 8.2: Properties of LoRA**

1. At initialisation the model is exactly the pretrained model.
2. The update $\Delta W = \frac{\alpha}{r} BA$ has rank $\Delta W \le r$.
3. It needs $r(d_{\text{in}} + d_{\text{out}})$ trainable numbers instead of $d_{\text{in}} d_{\text{out}}$.
4. After training it can be merged, $W = W_0 + \frac{\alpha}{r} BA$, so inference costs exactly the same as the original model.

*Proof.* (1) $B = 0 \Rightarrow \Delta W = 0$. (2) $\text{range}(BA) \subseteq \text{range}(B)$, which has dimension at most $r$. (3) Count entries of $A$ and $B$. (4) $W_0\boldsymbol{u} + \frac{\alpha}{r} BA\boldsymbol{u} = (W_0 + \frac{\alpha}{r} BA)\boldsymbol{u}$ by linearity. □

**Proposition 8.3: Gradients and the "B moves first" effect**

Let $\boldsymbol{g} = \partial \mathcal{L} / \partial \boldsymbol{o}$ be the gradient at the layer output $\boldsymbol{o} = W_0\boldsymbol{u} + \frac{\alpha}{r} BA\boldsymbol{u}$. Then

$$\frac{\partial \mathcal{L}}{\partial B} = \frac{\alpha}{r}\, \boldsymbol{g}\, (A\boldsymbol{u})^\top, \qquad \frac{\partial \mathcal{L}}{\partial A} = \frac{\alpha}{r}\, B^\top \boldsymbol{g}\, \boldsymbol{u}^\top.$$

At initialisation ($B = 0$) the gradient with respect to $A$ is zero, and only $B$ changes in the first step.

*Proof.* $\boldsymbol{o}$ depends on $B$ through $\frac{\alpha}{r} B\boldsymbol{v}$ with $\boldsymbol{v} = A\boldsymbol{u}$; $\partial(\boldsymbol{g}^\top B\boldsymbol{v})/\partial B = \boldsymbol{g}\boldsymbol{v}^\top$. Similarly $\boldsymbol{g}^\top BA\boldsymbol{u} = (B^\top \boldsymbol{g})^\top A\boldsymbol{u}$ gives $\partial / \partial A = B^\top \boldsymbol{g}\, \boldsymbol{u}^\top$. With $B = 0$ this vanishes. □

**Intuition: why low rank is enough**

Adapting a pretrained model to a related task seems to need only a few new "directions" in each layer: the pretrained features are already rich. The ratio $\alpha/r$ fixes the scale of the update so that changing $r$ does not require re-tuning the learning rate. Here $\alpha/r = 64/32 = 2$.

## 8.3 Where LoRA goes, and the exact parameter count

LoRA is applied to every attention projection (`q_proj, k_proj, v_proj, out_proj`) and both MLP matrices (`fc1, fc2`) in every encoder and decoder layer, with $r = 32$, $\alpha = 64$. In addition the bottom third of encoder layers is frozen: $\lfloor 32 \times 0.33 \rfloor = 10$ layers, so their LoRA matrices are not trained either (lower encoder layers are largely language-independent acoustic feature extractors; freezing them reduces over-fitting).

**Worked example 8.4: Counting the 86,880,008 "trainable" parameters of Stage 2**

Turbo: $d = 1280$, MLP width 5120. One LoRA pair on a $d_{\text{out}} \times d_{\text{in}}$ matrix costs $32(d_{\text{in}} + d_{\text{out}})$.

- Attention projection ($1280 \to 1280$): $32 \cdot 2560 = 81{,}920$. `fc1` or `fc2`: $32 \cdot (1280+5120) = 204{,}800$.
- Encoder layer: $4 \cdot 81{,}920 + 2 \cdot 204{,}800 = 737{,}280$. Trainable encoder layers: $32 - 10 = 22$, giving 16,220,160.
- Decoder layer: self-attention and cross-attention (8 projections) + MLP: $8 \cdot 81{,}920 + 2 \cdot 204{,}800 = 1{,}064{,}960$. Four layers: 4,259,840.
- LoRA total: **20,480,000**.
- Embedding matrix with the Odia row: $51{,}867 \times 1280 = 66{,}389{,}760$ (counted because `requires_grad=True`, although a gradient mask lets only one row of 1,280 numbers change; Chapter 9).
- LID-CTC head: $1280 \times 8 + 8 = 10{,}248$.

Sum: $20{,}480{,}000 + 66{,}389{,}760 + 10{,}248 = 86{,}880{,}008$, exactly the 86.9 M printed in the training log. The parameters that *actually* change are $20{,}480{,}000 + 1{,}280 + 10{,}248 \approx$ **20.5 M**, about 2.5% of the model.

**Check yourself 8.5: Whisper-small's 13.0 M**

Whisper-small: $d = 768$, MLP 3072, 12 encoder and 12 decoder layers, nothing frozen. Verify the 13.0 M trainable parameters of E1.
*Solution.* Attention pair: $32 \cdot 1536 = 49{,}152$; MLP pair: $32 \cdot 3840 = 122{,}880$. Encoder layer: $4 \cdot 49{,}152 + 2 \cdot 122{,}880 = 442{,}368$; decoder layer: $8 \cdot 49{,}152 + 2 \cdot 122{,}880 = 638{,}976$. Total $12(442{,}368 + 638{,}976) = 12{,}976{,}128 \approx 13.0$ M.

## 8.4 QLoRA

QLoRA stores the frozen $W_0$ in 4 bits (the NF4 data type: 16 quantisation levels placed at quantiles of a normal distribution, which suits roughly Gaussian weights, with per-block scales that are themselves quantised: "double quantisation"). Matrix products de-quantise to bfloat16 on the fly. Memory for $W_0$ drops about 4×; LoRA matrices stay in higher precision.

## 8.5 Experiment E1: LoRA vs QLoRA vs full fine-tuning

Whisper-small, Hindi only, 1,000 steps, batch 32; dev CER (FLEURS / IndicVoices / Kathbath). Untrained: 0.49 / 1.39 / 0.72.

| Method | trainable | time (min) | FLEURS | IndicVoices | Kathbath |
|---|---|---|---|---|---|
| LoRA ($r = 32$) | 13.0 M | 32 | 0.095 | **0.108** | **0.125** |
| QLoRA (NF4) | 13.0 M | 76 | **0.092** | 0.114 | 0.127 |
| Full fine-tune (lr $10^{-5}$) | 241.7 M | 28 | 0.101 | 0.173 | 0.143 |

- QLoRA matches LoRA but is 2.3× slower (de-quantisation overhead), and memory was not the constraint on a 122 GB machine.
- Full fine-tuning is clearly worse on the spontaneous IndicVoices set (0.173 vs 0.108), consistent with over-fitting the read-speech-heavy data and drifting from the pretrained acoustic model. (Caveat: its learning rate was not tuned.)

Decision: **LoRA**.

**In the code**

`train_whisper.py`: `LoraConfig(r=32, lora_alpha=64, lora_dropout=0.05, target_modules=[q_proj, k_proj, v_proj, out_proj, fc1, fc2])`, then `--freeze_enc_frac 0.33` sets `requires_grad=False` on all parameters (including LoRA) of encoder layers 0–9. `load_ft.py` loads the adapter and calls `merge_and_unload()` (Proposition 8.2(4)).

**Viva tip**

Be ready to say why the reported "87 M trainable" overstates what is learned: 66.4 M of it is the embedding matrix, of which only the Odia row can change. The honest figure is about 20.5 M.

Chapter 9

# Teaching Whisper a New Language Token: Odia

## 9.1 The problem

Odia has about 40 million speakers but no Whisper language token. Without a token we can neither force Odia when transcribing nor predict it as a language. We therefore *extend the vocabulary*.

## 9.2 Weight tying

**Definition 9.1: Tied input and output embeddings**

Whisper uses one matrix $\boldsymbol{E} \in \mathbb{R}^{V \times d}$ twice. As *input* embedding, token $v$ enters the decoder as the row $\boldsymbol{E}_v$. As *output* projection, the logits at a position with final hidden state $\boldsymbol{h}_t$ are $\boldsymbol{z}_t = \boldsymbol{E}\boldsymbol{h}_t$, i.e. $z_{t,v} = \boldsymbol{E}_v^\top \boldsymbol{h}_t$.

So adding a token means adding one row $\boldsymbol{E}_{V_0}$ (here $V_0 = 51{,}866$, the old vocabulary size), which acts both as the input vector of `<|or|>` and as its output "detector". Its gradient is the sum of both uses:

$$\nabla_{\boldsymbol{E}_{V_0}} \mathcal{L} = \underbrace{\sum_{t:\ \text{input}=\texttt{<|or|>}} \frac{\partial \mathcal{L}}{\partial \boldsymbol{e}_t}}_{\text{as input}} + \underbrace{\sum_t \left(p_{t,V_0} - \mathbb{1}[y_t = \texttt{<|or|>}]\right) \boldsymbol{h}_t}_{\text{as output (Prop. 4.2)}} .$$

The second sum runs over *every* position of *every* utterance: wherever `<|or|>` is not the target, the row is pushed away from $\boldsymbol{h}_t$ in proportion to the probability it wrongly receives.

## 9.3 Initialisation from Bengali

**Definition 9.2: The Odia row**

After `resize_token_embeddings`$(V_0 + 1)$, set $\boldsymbol{E}_{V_0} \leftarrow \boldsymbol{E}_{\text{id}(\texttt{<|bn|>})}$.

**Proposition 9.3: The clone starts as a perfect tie**

Immediately after initialisation, for every input and every position, $z_{t,\texttt{<|or|>}} = z_{t,\texttt{<|bn|>}}$. Hence the closed-set LID probabilities of Odia and Bengali are equal, and an Odia-prompted decode is identical to a Bengali-prompted decode.

*Proof.* Both logits equal $\boldsymbol{E}_v^\top \boldsymbol{h}_t$ with identical rows. As inputs the two tokens produce identical embeddings, so all hidden states are identical too. □

**Intuition: why Bengali**

Odia, Bengali and Assamese are Eastern Indo-Aryan languages with similar sound systems, and the Odia script is historically related to the Bengali–Assamese script. Starting as "Bengali" means Odia audio immediately gets a high language score, and the decoder starts from

a conditioning it already knows (an Eastern Indo-Aryan language, not English). Training then only has to *separate* Odia from Bengali, which the tie in Proposition 9.3 forces it to do: only the Odia row can move.

## 9.4 Training only one row: the gradient mask

LoRA does not touch the embedding, so by default it is frozen. We set `requires_grad=True` on $\boldsymbol{E}$ and register a hook that multiplies its gradient by a mask $M$ whose only non-zero row is row $V_0$:

$$\nabla_{\boldsymbol{E}}\mathcal{L} \;\leftarrow\; M \odot \nabla_{\boldsymbol{E}}\mathcal{L}, \qquad M_{v,:} = \mathbb{1}[v = V_0].$$

**Proposition 9.4: AdamW leaves zero-gradient rows unchanged if and only if weight decay is off**

Consider AdamW with moments initialised to 0. If a parameter receives gradient 0 at every step, then its Adam update is 0 at every step. Its total change is $-\eta\lambda\theta$ per step from decoupled weight decay, which is 0 if and only if $\lambda = 0$ (or $\theta = 0$).

*Proof.* Adam: $m_s = \beta_1 m_{s-1} + (1-\beta_1) g_s$, $v_s = \beta_2 v_{s-1} + (1-\beta_2) g_s^2$. With $g_s = 0$ and $m_0 = v_0 = 0$, induction gives $m_s = v_s = 0$, so the step $\eta\,\hat{m}_s/(\sqrt{\hat{v}_s} + \epsilon) = 0/\epsilon = 0$. AdamW then applies $\theta \leftarrow \theta - \eta\lambda\theta$ separately, regardless of the gradient. □

Therefore the embedding is put in its own optimiser group with **weight decay 0**. With decay, all 51,866 frozen rows (including every text token of every language) would slowly shrink toward zero.

**In the code**

`train_whisper.py`:

```
LANGID["or"] = V0
model.resize_token_embeddings(V0 + 1)
E[V0] = E[LANGID["bn"]].clone()
row_mask = torch.zeros_like(emb); row_mask[V0] = 1
emb.register_hook(lambda g: g * row_mask.to(g.dtype))
groups.append({"params": [emb], "weight_decay": 0.0, "lr": a.emb_lr})
```

PEFT saves the resized `embed_tokens`/`proj_out` with the adapter (verified by reloading: the Odia row is restored). `load_ft.py` detects "adapter vocabulary = base +1" and resizes the base model *before* loading.

## 9.5 Choosing the learning rate of the row

The Odia row is a single vector trained from scratch-ish, so its learning rate was chosen by a small sweep (400 steps, Whisper-small, `runs/embchk_*`):

| `emb_lr` | $5 \times 10^{-5}$ | $\mathbf{2.5 \times 10^{-4}}$ | $1.5 \times 10^{-3}$ |
|---|---|---|---|
| Odia token-LID | 0.74 | 0.88 | 0.90 |
| Other languages | unchanged | unchanged | unchanged |

$2.5 \times 10^{-4}$ gives almost the LID of the largest value with less drift. (An early all-Odia LID seen at step 40 of a turbo smoke test was first blamed on the large learning rate; the sweep showed it

was a transient. The log records this wrong suspicion.)

## 9.6 A silent trap: `convert_tokens_to_ids`

**Viva tip: bug 15**

`tok.convert_tokens_to_ids("<|or|>")` does not raise an error. The tokenizer does not know the string, so it returns the *unknown* id, which for Whisper is 50257 `<|endoftext|>`. An evaluation script that prompted Odia this way would have prompted with EOT and silently produced empty or garbage output. All code now takes the id from `LANGID` returned by `load_ft.load`.

## 9.7 Result

Test split, Stage 2 (Chapter 14):

| Set | vanilla turbo CER | Stage 2 CER (oracle) | Stage 2 token-LID |
|---|---|---|---|
| FLEURS or | 1.023 | 0.145 | 1.00 |
| Kathbath or | 1.299 | 0.041 | 1.00 |
| IndicVoices or | 2.046 | 0.196 | 0.75 |

The new token works. One side effect: the `<|or|>` column of the confusion matrix absorbs errors from other languages (17 Hindi, 22 Assamese, 15 English, 11 Bengali test utterances by token LID). A plausible explanation (not tested) is that `<|or|>` is the only output row allowed to change, so it is the cheapest way for the optimiser to raise any language score, and it may drift toward a generic "Indic" direction.

**Check yourself 9.5: Why not train the whole embedding matrix?**

*Solution.* 66 M extra free parameters, all text-token rows of all languages would drift (forgetting), and every row would need weight decay decisions. The tied output projection means drifting rows also change every logit. One masked row adds the new language with the least possible disturbance.

Chapter 10

# The Encoder LID-CTC Head: Connectionist Temporal Classification

The decoder's LID uses only one softmax at the first decoder step. The literature (ML-SUPERB 2.0, Interspeech 2025) found that an *auxiliary LID loss on the encoder* improves LID markedly. This chapter builds the CTC loss from scratch, specialises it to our one-label case, and shows how it is attached to Whisper.

## 10.1 CTC in general

CTC (Graves et al., 2006) trains a frame-wise classifier when we know the label *sequence* but not which frames correspond to which label.

**Definition 10.1: Alignments and the collapse map**

Let $\mathcal{C}$ be the label set and $\varnothing$ a special *blank*. An *alignment* of length $T$ is $\boldsymbol{\pi} = (\pi_1, \dots, \pi_T) \in (\mathcal{C} \cup \{\varnothing\})^T$. The collapse map $\mathcal{B}$ first merges consecutive repeats, then deletes blanks. Example: $\mathcal{B}(\varnothing\, a\, a\, \varnothing\, a\, b\, b) = \mathcal{B}(\varnothing\, a\, \varnothing\, a\, b) = (a, a, b)$.

**Definition 10.2: CTC probability and loss**

A network outputs, at every frame $t$, a distribution $p_t(\cdot)$ over $\mathcal{C} \cup \{\varnothing\}$, with frames conditionally independent given the input. Then

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \sum_{\boldsymbol{\pi} \in \mathcal{B}^{-1}(\boldsymbol{y})} \prod_{t=1}^{T} p_t(\pi_t), \qquad \ell_{\mathrm{CTC}}(\boldsymbol{x}, \boldsymbol{y}) = -\log p(\boldsymbol{y} \mid \boldsymbol{x}).$$

The blank does two jobs: it lets frames say "nothing new here", and it separates genuine repeats ($a\, \varnothing\, a \mapsto aa$, but $a\, a \mapsto a$).

**Theorem 10.3: Forward algorithm**

Let $\boldsymbol{y}' = (\varnothing, y_1, \varnothing, y_2, \dots, y_U, \varnothing)$ (length $2U + 1$). Define $\alpha_t(s)$ as the total probability of all prefixes $\pi_{1..t}$ that collapse to the part of $\boldsymbol{y}'$ ending at position $s$. Then $\alpha_1(1) = p_1(\varnothing)$, $\alpha_1(2) = p_1(y_1)$, $\alpha_1(s) = 0$ for $s > 2$, and

$$\alpha_t(s) = \Big(\alpha_{t-1}(s) + \alpha_{t-1}(s-1) + \mathbb{1}\big[y'_s \neq \varnothing,\ y'_s \neq y'_{s-2}\big]\, \alpha_{t-1}(s-2)\Big)\, p_t(y'_s),$$

with $p(\boldsymbol{y} \mid \boldsymbol{x}) = \alpha_T(2U + 1) + \alpha_T(2U)$. Cost: $O(TU)$ instead of a sum over exponentially many alignments.

*Proof sketch.* A valid path moves through $\boldsymbol{y}'$ left to right; at each frame it may stay on the same symbol, advance by one, or skip a blank between two *different* labels (skipping between equal labels would merge them). Summing over the three predecessors and multiplying by the current frame's probability gives the recursion; a complete path must end on the last label or the final blank. □

## 10.2 The one-label special case

For LID the "label sequence" of an utterance is just its language: $\boldsymbol{y} = (\ell)$, $U = 1$.

**Proposition 10.4: Closed form of single-label CTC**

For $\boldsymbol{y} = (\ell)$,

$$\mathcal{B}^{-1}\big((\ell)\big) = \big\{\, \varnothing^a\, \ell^b\, \varnothing^c \;:\; a, c \geq 0,\ b \geq 1,\ a + b + c = T \,\big\},$$

a set of $T(T+1)/2$ alignments, and

$$p(\ell \mid \boldsymbol{x}) = \sum_{1 \leq s \leq e \leq T} \prod_{t<s} p_t(\varnothing) \prod_{t=s}^{e} p_t(\ell) \prod_{t>e} p_t(\varnothing).$$

*Proof.* $\mathcal{B}(\boldsymbol{\pi}) = (\ell)$ requires that no other language appears (it would survive the collapse) and that, after merging repeats, exactly one $\ell$ remains. If $\ell$ appeared in two runs separated by a blank, the collapse would give $(\ell, \ell)$. So the $\ell$'s form one contiguous run $\pi_s..\pi_e$ with $1 \leq s \leq e \leq T$, blanks elsewhere. The number of pairs $s \leq e$ is $\binom{T}{2} + T = T(T+1)/2$. The probability is the sum of the path products. □

**Intuition: what the head learns**

To make $p(\ell \mid \boldsymbol{x})$ large, the head must output "$\ell$" with high probability on *one stretch* of frames and "blank" elsewhere, and must give the *other* languages low probability everywhere (any frame on another language ruins every path through it). It is a "detect the language somewhere" objective, a soft version of multiple-instance learning. The gradient reaches *every* encoder frame directly, rather than only through the decoder's first step.

## 10.3 Attaching the head to Whisper

**Definition 10.5: LID-CTC head**

A single linear layer $\boldsymbol{u} \mapsto W\boldsymbol{u} + \boldsymbol{c}$, $W \in \mathbb{R}^{8 \times 1280}$, on each encoder output $\boldsymbol{h}_t$, followed by a softmax over 8 classes: blank (class 0) and the 7 languages in alphabetical order (as, bn, en, hi, mr, or, te). It is trained with

$$\mathcal{L}_{\text{LID}} = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \frac{\ell_{\text{CTC}}\big(\boldsymbol{x}^{(i)}, (\ell^{(i)})\big)}{T_i},$$

where $\mathcal{S}$ is the set of *speech* clips in the batch (non-speech clips are excluded, Chapter 11) and $T_i$ is the number of valid encoder frames (Proposition 2.6).

**Why divide by $T_i$?** Bug 9: without it the loss was about 300 at initialisation. A randomly initialised head assigns roughly $1/8$ to each class, so every path has probability about $8^{-T}$ and $-\log p(\ell \mid \boldsymbol{x}) \approx T \log 8 - \log \frac{T(T+1)}{2}$, which grows linearly in $T$ (hundreds for clips of several seconds). That would swamp the token cross-entropy (about 2 at the start). Dividing by $T_i$ makes it a per-frame quantity on the same scale. This was fixed before any experiment ran.

**Definition 10.6: The joint objective**

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{CE}}(\theta) + \lambda\, \mathcal{L}_{\text{LID}}(\theta), \qquad \lambda = 0.3.$$

The head has its own learning rate $10^{-3}$. Its gradient flows into the encoder (it is not detached).

## 10.4 From frame posteriors to one decision

At test time the head gives $p_t(c)$ for every valid frame. The utterance decision is

$$\hat{\ell}_{\text{head}} = \arg\max_{\ell \in \mathcal{L}} \sum_{t=1}^{T} p_t(\ell),$$

blank excluded. The normalised vector $q(\ell) = \sum_t p_t(\ell) / \sum_{\ell'} \sum_t p_t(\ell')$ is used as the head's probability in the calibrated rule of Chapter 15.

**In the code**

`train_whisper.py`: `F.ctc_loss(log_probs, lidx.unsqueeze(1), nfr, ones, blank=0, zero_infinity=True, reduction="none") / nfr`, masked to speech clips. Target length 1, input length `nfr` $= T_i$. `common.py`: `head_lid` implements the summed-posterior decision; `lid_head.pt` stores the class order.

## 10.5 Experiment E2: how much LID-CTC?

Whisper-small, LoRA, hi/te/as, 1,200 steps, $\lambda \in \{0, 0.1, 0.3, 1.0\}$; dev split.

| | $\lambda = 0$ | 0.1 | **0.3** | 1.0 |
|---|---|---|---|---|
| CER IndicVoices as | 0.246 | 0.250 | 0.246 | **0.358** |
| CER FLEURS as | 0.204 | 0.140 | 0.158 | 0.178 |
| CER IndicVoices hi | 0.132 | 0.131 | 0.122 | 0.129 |
| token-LID IndicVoices as | 0.73 | 0.81 | 0.84 | 0.83 |
| token-LID IndicVoices hi | 0.86 | 0.84 | 0.83 | 0.84 |
| head-LID IndicVoices hi | – | 0.91 | 0.92 | 0.91 |

- CER is unchanged within noise, except that $\lambda = 1$ clearly hurts Assamese IndicVoices: too much weight on LID competes with transcription.
- The auxiliary loss *improves the decoder token LID* on the hard Assamese set (0.73 → 0.81–0.84): a shared-encoder effect.
- On hard sets the head itself is better than the token (Hindi IndicVoices 0.92 vs 0.83).

Decision: $\lambda = 0.3$ (the evidence is weak; differences between 0.1 and 0.3 are within noise).

**At scale (Stage 2, test).** The head beats the token most where LID is hardest: IndicVoices Assamese token 0.66, head 0.83. Pooled recall of Assamese: token 0.82, head 0.92 (Chapter 14).

**Check yourself 10.7: A 3-frame example**

$T = 3$, true language $\ell$. Frame posteriors: $p_t(\ell) = (0.2, 0.7, 0.6)$, $p_t(\varnothing) = (0.7, 0.2, 0.3)$ (rest on other languages). Compute $p(\ell \mid \boldsymbol{x})$ with Proposition 10.4.
*Solution.* Sum over runs $[s, e]$: $[1,1]$: $0.2 \cdot 0.2 \cdot 0.3 = 0.012$; $[2,2]$: $0.7 \cdot 0.7 \cdot 0.3 = 0.147$; $[3,3]$: $0.7 \cdot 0.2 \cdot 0.6 = 0.084$; $[1,2]$: $0.2 \cdot 0.7 \cdot 0.3 = 0.042$; $[2,3]$: $0.7 \cdot 0.7 \cdot 0.6 = 0.294$; $[1,3]$:

$0.2 \cdot 0.7 \cdot 0.6 = 0.084$. Total 0.663; loss $-\log 0.663 = 0.411$, per frame 0.137. The utterance decision uses $\sum_t p_t(\ell) = 1.5$.

Chapter 11

# Robustness: Augmentation, Non-Speech Clips and Hallucination

## 11.1 Why not a denoiser?

The obvious idea, cleaning the audio with a speech-enhancement network before recognition, was rejected on evidence: three independent 2025–26 studies found that enhancement in front of a strong ASR model raises WER (MetricGAN+ degraded ASR in all 40 tested configurations; SAM-Audio raised Whisper's WER/CER on noisy Bengali). Enhancement optimises *perceptual* quality and introduces artefacts the recogniser has never seen. The alternative is to expose the recogniser itself to degraded audio during training.

## 11.2 The four augmentations

All augmentation is applied on the fly, per sample, with a seeded random generator (Chapter 12).

### 11.2.1 Additive noise at a chosen SNR

**Definition 11.1: Signal-to-noise ratio**

For signal $\boldsymbol{s}$ and noise $\boldsymbol{n}$ of equal length, with powers $P_s = \frac{1}{N}\sum s_i^2$ and $P_n = \frac{1}{N}\sum n_i^2$, $\mathrm{SNR} = 10\log_{10}(P_s/P_n)$ dB.

**Proposition 11.2: Mixing at a target SNR**

To mix noise into $\boldsymbol{s}$ at exactly $\sigma$ dB, use $\boldsymbol{s} + g\boldsymbol{n}$ with $g = \sqrt{P_s/(P_n 10^{\sigma/10})}$.

*Proof.* The scaled noise has power $g^2 P_n = P_s 10^{-\sigma/10}$, so $10\log_{10}\left(P_s/(g^2 P_n)\right) = \sigma$. □

Training noise comes from MUSAN *noise* and *music* (speech was deliberately excluded), at $\sigma \sim \mathcal{U}(0, 20)$ dB. At 0 dB noise is as loud as speech.

### 11.2.2 Reverberation

A room is (approximately) a linear time-invariant system, fully described by its *room impulse response* (RIR) $\boldsymbol{r}$. The reverberant signal is the convolution $y[n] = \sum_k r[k]\, s[n-k]$, computed with an FFT, truncated to the original length and rescaled to the original energy.

**Viva tip: bug 11, a contaminated RIR pool**

92 of the 2,417 files in the "real RIRs" folder were 30-s, 8-channel *isotropic noise* recordings, not impulse responses, and multi-channel RIRs were being averaged across microphones (different delays smear the response). So about 4% of "reverb" augmentations convolved speech with noise. Fixed: noise files excluded (2,325 RIRs remain), first channel only. The reverb columns of the early E4 robustness results used the contaminated pool.

### 11.2.3 Speed perturbation

Resampling by a rational factor $u/d$ and playing at 16 kHz changes duration by $u/d$ and pitch by $d/u$. Factors: 10/9, 11/10 (slower) or 9/10, 10/11 (faster), i.e. about ±10%.

**Proposition 11.3: The 27-second guard**

The largest stretch is 10/9. A clip stays within Whisper's 30-s window after stretching iff its duration is at most $30 \cdot 9/10 = 27$ s.

Bug 12: before the guard, 228 of the 349,592 mix clips could be stretched past 30 s. The feature extractor then cut the audio but the label kept the full text, teaching the model to invent words that were never heard. Now clips $\geq 27$ s are never speed-perturbed.

### 11.2.4 SpecAugment

On the log-mel spectrogram, per sample: two time masks of width $\sim \mathcal{U}\{0, .., 39\}$ frames (up to 0.39 s) and two frequency masks of width $\sim \mathcal{U}\{0, .., 9\}$ mel bins, filled with the sample's minimum value. Bug 14: the masks used to be drawn over the whole 30-s window, so for short clips most masks fell on padding. They are now placed inside the valid length ($2T$ mel frames).

**Worked example 11.4: How often is a sample clean?**

`augment.py`: speed with probability 0.3 (if $< 27$ s), reverb 0.2, noise 0.5, independently. $\mathbb{P}(\text{no waveform augmentation}) = 0.7 \cdot 0.8 \cdot 0.5 = 0.28$. SpecAugment is applied to every sample when `--aug` is on.

## 11.3 Hallucination and the non-speech recipe

**Definition 11.5: Hallucination and false silence**

On a clip that contains no speech (noise or music only), the model *hallucinates* if it outputs more than 5 tokens. On a clip that contains speech, *false silence* means an empty output. Both are measured on held-out files.

Whisper is notorious for producing fluent text on silence or music ("Thank you for watching"). The Canary and Calm-Whisper recipe: show the model non-speech audio paired with an *empty* transcript.

**Definition 11.6: Non-speech training clips**

With probability 0.05 a training sample is replaced by a random 2–12 s excerpt of a training noise or music file, with

$$\text{decoder input} = [\texttt{SOT},\ \texttt{<|}\ell_{\text{rand}}\texttt{|>},\ \texttt{<|transcribe|>},\ \texttt{<|notimestamps|>}]$$
$$\text{labels} = [\ -100,\ \texttt{<|transcribe|>},\ \texttt{<|notimestamps|>},\ \texttt{EOT}\ ]$$

where $\ell_{\text{rand}}$ is a uniformly random language. The language label is masked ($-100$) and the clip is excluded from the LID-CTC loss.

**Intuition: why mask the language**

Noise has no language. If the random language were a target, the model would be trained to predict a random answer, which is pure label noise for both LID outputs (bug 13). What *is*

trained is the crucial decision "after the prompt, emit EOT immediately".

## 11.4 A metric that could not go below 100%

**Viva tip: bug 10, the most instructive mistake of the project**

The first hallucination metric reported 97–100% for *every* model, including one trained on blank-label noise. The first diagnosis was "one EOT target per noise clip is too weak a signal compared with ∼100 target tokens per speech clip", and a 30× loss weight on that EOT was added. **That diagnosis was wrong.** HuggingFace Whisper's default `begin_suppress_tokens`= [220, 50257] sets the logits of a space and of EOT to $-\infty$ *at the first generated position*. An empty transcript was therefore impossible, and the metric had a floor near 100%. With suppression disabled, the original unweighted model hallucinated on only 2–4% of clips. Lesson: when a metric does not move at all, suspect the measurement before the model.

## 11.5 Experiment E4

Whisper-small, LoRA, hi/te/as, 1,200 steps.

**Clean dev CER** is about the same with and without augmentation (e.g. FLEURS as 0.204 → 0.170, FLEURS hi 0.094 → 0.114, IndicVoices as 0.246 → 0.278: mixed, within noise).

**Noisy test audio** (held-out noise files, Hindi FLEURS, CER):

| Model | clean | 20 dB | 15 | 10 | 5 | 0 dB | reverb |
|---|---|---|---|---|---|---|---|
| untrained Whisper-small | 0.357 | 0.478 | 0.548 | 0.588 | 0.703 | 0.806 | 0.547 |
| LoRA, no augmentation | 0.098 | 0.102 | 0.113 | 0.129 | 0.165 | 0.367 | 0.231 |
| LoRA + aug + non-speech | 0.097 | 0.103 | 0.108 | 0.162 | 0.155 | **0.217** | 0.257 |

The benefit appears mainly at low SNR (0 dB: 0.367 → 0.217).

**Hallucination** (100 held-out noise/music clips per language, suppression disabled):

| Model | hi | te | as |
|---|---|---|---|
| untrained Whisper-small | 0.98 | 1.00 | 1.00 |
| LoRA, no non-speech data | 0.98 | 1.00 | 0.98 |
| LoRA + aug + non-speech (unweighted EOT) | **0.04** | **0.02** | **0.03** |
| LoRA + aug + non-speech (EOT weight 30) | 0.00 | 0.00 | 0.00 |

Mean output tokens on noise fell from 150–180 to 3–7. False silence on real speech stayed at 0.00 (0.01 for Assamese with weight 30). Decision: keep augmentation and non-speech clips, *unweighted* (weight 30 was never needed; it slightly raises false silence).

**At scale (Stage 2):** hallucination 0.00–0.01 in all 7 languages, false silence 0.00 (Chapter 14).

**In the code**

`augment.py`: `add_noise`, `reverb`, `speed`, `augment`, `spec_augment`. Noise and RIR files are split by an MD5 hash of the path: hash $\bmod 5 = 0$ (20%) held out for evaluation, the rest for training, so no evaluation noise file is ever heard in training. `eval_halluc.py` sets `begin_suppress_tokens=None`. The `hallucination_rate` fields in old `snr_*.json` files are invalid and must be ignored.

**Check yourself 11.7: Why use a hash of the path rather than random.shuffle for the 80/20 split?**

*Solution.* The hash split is a deterministic function of the file name, so every script, run and machine gets exactly the same partition without sharing a seed or a file list, and adding new files never moves old ones between the two sides.

Chapter 12

# The Training Procedure

## 12.1 The full loss of one batch

A batch has 32 samples (about 5% of them non-speech). With the label conventions of Chapters 5 and 11:

$$\mathcal{L} = \underbrace{\frac{\sum_{i,t} \omega_{it} \mathbb{1}[y_{it} \neq -100] \left( -\log p_\theta(y_{it} \mid \cdot) \right)}{\sum_{i,t} \mathbb{1}[y_{it} \neq -100]}}_{\text{token cross-entropy}} + \lambda \underbrace{\frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \frac{\ell_{\text{CTC},i}}{T_i}}_{\text{LID-CTC}},$$

with $\omega_{it} = 1$ except $\omega_{i,3} = w_{\text{ns}}$ on the EOT of a non-speech sample ($w_{\text{ns}} = 1$ in Stage 2), and $\lambda = 0.3$.

## 12.2 The optimiser

**Definition 12.1: AdamW**

For each parameter $\theta$ with gradient $g_s$ at step $s$:

$$m_s = \beta_1 m_{s-1} + (1-\beta_1) g_s, \qquad v_s = \beta_2 v_{s-1} + (1-\beta_2) g_s^2,$$
$$\hat{m}_s = m_s / (1-\beta_1^s), \qquad \hat{v}_s = v_s / (1-\beta_2^s),$$
$$\theta \leftarrow \theta - \eta_s \Big( \frac{\hat{m}_s}{\sqrt{\hat{v}_s} + \epsilon} + \lambda_{\text{wd}} \theta \Big).$$

Here $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-6}$; weight decay 0 for LoRA, the embedding and the head.

The bias corrections $1/(1-\beta^s)$ undo the bias toward 0 caused by initialising the moments at 0. Dividing by $\sqrt{\hat{v}}$ gives every parameter a step of roughly size $\eta$ regardless of its gradient scale.

Three parameter groups with different learning rates:

| Group | peak learning rate | weight decay |
|---|---|---|
| LoRA matrices (22 encoder + 4 decoder layers) | $3 \times 10^{-4}$ | 0 |
| Embedding (only the `<\|or\|>` row can change) | $2.5 \times 10^{-4}$ | 0 |
| LID-CTC head | $10^{-3}$ | 0 |

## 12.3 Learning-rate schedule

**Definition 12.2: Linear warm-up, linear decay**

With $S$ total steps and $W = \max(1, 0.1S)$ warm-up steps, every group's learning rate is its

peak times

$$\phi(s) = \begin{cases} \dfrac{s+1}{W}, & s < W, \\ \max\Big(0,\ \dfrac{S-s}{\max(1, S-W)}\Big), & s \geq W. \end{cases}$$

For Stage 2, $S = 11{,}000$ and $W = 1{,}100$. Warm-up matters for Adam: in the first steps $\hat{v}$ is estimated from very few gradients and can be tiny, which would make the first updates huge.

## 12.4 Gradient clipping and precision

**Clipping.** If the global gradient norm $\|g\| = \big(\sum_{\text{all params}} \|g_\theta\|^2\big)^{1/2}$ exceeds 1, all gradients are rescaled by $1/\|g\|$. This bounds the size of any single update (a single bad batch cannot throw the model far away).

**bfloat16 autocast.** Matrix products run in bfloat16 (8-bit exponent like float32, 7-bit mantissa), halving memory traffic with the same numerical range as float32; the loss and softmax are computed in float32.

**`torch.compile`.** The forward pass is compiled into fused kernels; it gave 12.7 → about 23 utterances/s in a benchmark (15–17 in real Whisper-small runs). To limit the number of distinct shapes that must be compiled, decoder sequences are padded to a multiple of 64 tokens.

## 12.5 Reproducible randomness and resumable training

**Definition 12.3: Seeded data order**

Epoch $e$ uses the permutation `RandomState(seed+`$e$`).permutation(n)`; batch $b$ is positions $32b..32b + 31$. Sample $i$ in epoch $e$ draws all its augmentation randomness from `Random(f"{seed}:{e}:{i}")`. SpecAugment has its own generator.

Because every random choice is a function of (seed, epoch, index), the data a run sees does not depend on how many DataLoader workers there are or in which order they finish, and a resumed run sees exactly the same samples as an uninterrupted one.

**In the code**

A checkpoint (every 500 steps, written atomically via a temporary file and `os.replace`) stores: trainable parameters, head, optimiser and scheduler states, step, epoch and batch position, the SpecAugment and CUDA random states (CUDA RNG drives LoRA dropout), loss history, partial results and elapsed time. `--resume` restores all of it. Verified: resuming 20 → 40 steps differs from an uninterrupted run by 0.004 in loss, less than two identical fresh runs differ from each other (0.009–0.014, GPU non-determinism).

## 12.6 Stage 2 in numbers

| | |
|---|---|
| Base model | Whisper large-v3-turbo |
| Data | 700 h mix (349,592 references), 7 languages + MUCS |
| Steps × batch | 11,000 × 32 = 352,000 samples ≈ 1 epoch |
| LoRA | $r = 32$, $\alpha = 64$, dropout 0.05, encoder layers 0–9 frozen |
| Extras | `<\|or\|>` row, LID-CTC $\lambda = 0.3$, augmentation, 5% non-speech |
| Throughput | 5.9 utterances/s; 59,895 s = 16.6 h; peak memory 88.6 GB |
| Final training loss | 0.159 (mean of last 50 steps) |

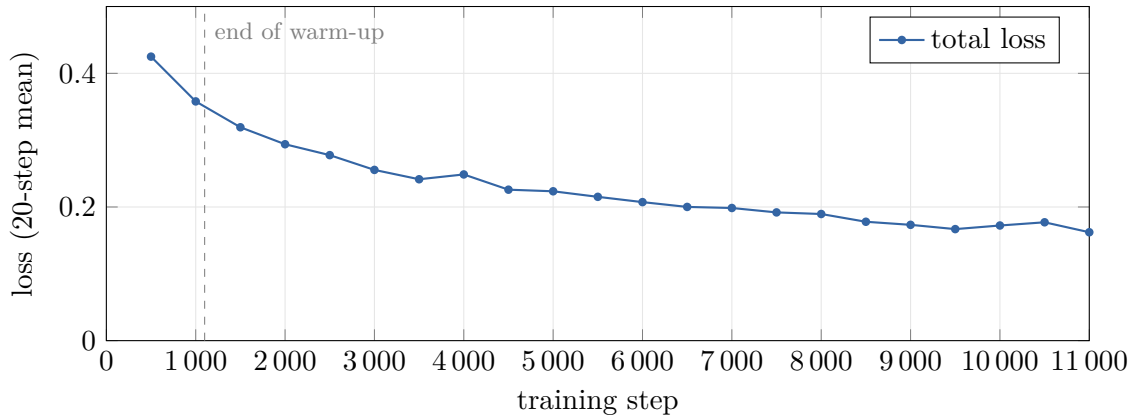



Figure 12.1: Stage 2 training loss (from `logs/s2_turbo.log`; step 1 loss was 2.13, off the chart). The LID-CTC term fell from 2.04 at step 1 to about 0.003 by step 1,000 and 0.001 at the end: the head learns quickly, and afterwards almost all of the loss is transcription.

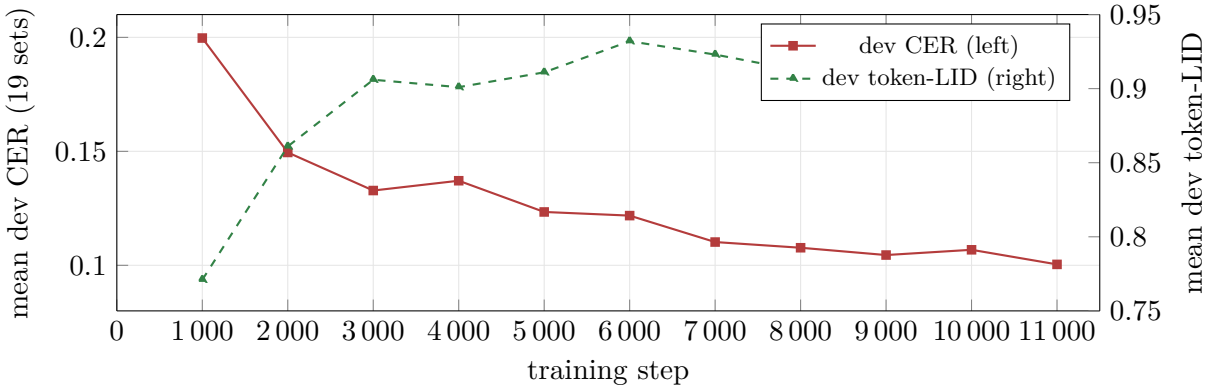



Figure 12.2: Dev-set evaluation inside the Stage 2 run (48 utterances per set, `runs/s2_turbo/result.json`). CER was still falling at the last evaluation (0.105 at step 9,000, 0.100 at the end), while LID plateaued around 0.93 after 6,000 steps.

**Check yourself 12.4: Learning rate at step 6050**

*Solution.* $s = 6050 \geq W = 1100$: $\phi = (11000 - 6050)/(11000 - 1100) = 4950/9900 = 0.5$. LoRA learning rate $1.5 \times 10^{-4}$, Odia row $1.25 \times 10^{-4}$, head $5 \times 10^{-4}$.

**Viva tip**

Why only one epoch? Compute: one epoch is 16.6 h on the only GPU, and the plan was to evaluate before spending more. Note also that the dev CER evaluated every 1,000 steps was

still improving slowly at the end, so more training (on a reseeded mix) might help; this is listed as future work (see the dev curve above: 0.200 at step 1,000, 0.100 at the end, not yet flat).

Chapter 13

# Evaluation Protocol, Baselines and Statistics

## 13.1 The test protocol

- **23 test sets**: every (corpus, language) pair in the test split: FLEURS ×7, IndicVoices ×6, Kathbath ×5, Lahaja hi, LibriSpeech en, Svarah en, MUCS hi and bn.
- **100 utterances per set**, sampled with `random.Random(0)` from the test split. The *same* 2,300 utterances are used for every system, which makes paired comparisons possible.
- Greedy decoding, at most 440 new tokens, the shared normaliser, micro CER per set.
- Summary: the plain mean of the per-set CERs, plus the worst set and the standard deviation (the ML-SUPERB 2.0 style).

## 13.2 Baselines

**Vanilla Whisper large-v3-turbo.** The starting point. Odia is decoded without a language token (it has none).

**IndicConformer-600M (AI4Bharat).** A Conformer encoder (convolution + self-attention) with a CTC head over per-language character vocabularies, trained on large AI4Bharat corpora (the same sources as IndicVoices and Kathbath). It has no English and no LID: the language is always given. It is decoded greedily, which for CTC is simple:

**Definition 13.1: Greedy CTC decoding**

Take $\hat{\pi}_t = \arg\max_c p_t(c)$ at every frame, then output $\mathcal{B}(\hat{\pi})$ (merge repeats, drop blanks).

This is exactly what `ic_gpu.py` does (`unique_consecutive`, drop id 256 = blank). The ONNX runtime had no GPU kernels for this GPU (bug 6), so the encoder and CTC head were converted to PyTorch with `onnx2torch` (verified: 23 of 24 test transcripts identical to the CPU ONNX path, the last differs by one trailing character from batch padding).

**Qwen3-ASR 0.6B and 1.7B.** LLM-based ASR with built-in LID for 52 languages, but of ours it supports only Hindi and English. On other languages it answers "Hindi" or an unrelated language and its CER is 0.34–1.0, so it is compared only on the Hindi and English sets, with the language forced.

**SpeechBrain ECAPA-TDNN (VoxLingua107).** A dedicated 107-language LID model, used closed-set over the 6 of our languages it knows (it has no Odia class). This is the external LID baseline.

## 13.3 Statistics: how sure are we?

A CER computed on 100 utterances is a random quantity: another 100 utterances would give a different number. We need error bars.

**Definition 13.2: Percentile bootstrap confidence interval for CER**

For a test set with per-utterance edit counts $e_i$ and reference lengths $n_i$, $i = 1..K$, $\widehat{\text{CER}} = \sum_i e_i / \sum_i n_i$. For $b = 1..B$ ($B = 2000$), draw $K$ indices $i_1^{(b)}..i_K^{(b)}$ uniformly *with replacement* and compute $\text{CER}^{*(b)} = \sum_j e_{i_j^{(b)}} / \sum_j n_{i_j^{(b)}}$. The 95% interval is $[q_{0.025}, q_{0.975}]$ of $\{\text{CER}^{*(b)}\}$.

**Intuition: why resample utterances, not characters**

Errors inside one utterance are strongly correlated (a misrecognised word costs several characters; a loop costs hundreds). The independent unit is the utterance, so that is what we resample. Resampling the pair $(e_i, n_i)$ together keeps the ratio structure of CER.

**Definition 13.3: Paired bootstrap test**

For two systems A and B scored on the same utterances, use the *same* resampled indices for both and record $\Delta^{*(b)} = \text{CER}_A^{*(b)} - \text{CER}_B^{*(b)}$. The difference is called significant at 5% if the 95% percentile interval of $\Delta^*$ excludes 0.

**Proposition 13.4: Pairing reduces variance**

$\text{Var}(\hat{C}_A - \hat{C}_B) = \text{Var}(\hat{C}_A) + \text{Var}(\hat{C}_B) - 2\,\text{Cov}(\hat{C}_A, \hat{C}_B)$. When both systems find the same utterances hard (positive covariance), the paired variance is smaller than for independent samples.

*Proof.* Bilinearity of covariance. □

Hard utterances (noisy, fast, rare words) are hard for every system, so the covariance is large and the paired test detects differences that two separate CIs would not.

**Worked example 13.5: Reading the numbers**

With 100 utterances a single cell has roughly ±0.01–0.03 CER uncertainty. IndicConformer minus Stage 2 on FLEURS Telugu: +0.001, paired 95% interval $[-0.015, +0.016]$, which contains 0: "not significantly different". On IndicVoices Assamese: −0.110, interval $[-0.136, -0.088]$: IndicConformer is significantly better. FLEURS Hindi is a borderline case: −0.012, interval $[-0.024, -0.0002]$, which just excludes 0.

**Viva tip: what the bootstrap does *not* cover**

It quantifies *which test utterances were sampled*. It says nothing about training randomness (seed variance): only one seed was trained, because one Stage 2 run costs 16.6 GPU hours. Say this unprompted if results are questioned.

## 13.4 Code-switched speech needs a different metric

MUCS references write English words sometimes in Latin script ("computer") and sometimes in Devanagari (कंप्यूटर); models also choose either. Plain CER counts "computer" vs कंप्यूटर as entirely wrong, although both are correct transcriptions.

**Definition 13.6: Transliteration-tolerant scores `cs_metric.py`**

**rom**: normalise, then romanise *every* Indic script with ISO 15919 (a hypothesis may be in another script), map anusvara to $n$ ($m$ before $p/b$), strip diacritics, lower-case, drop a word-final inherent $a$. **skel**: per word, keep the first letter and drop later vowels and $h, y, w$; fold $c/q/x/z/ph$; collapse repeated letters.

$$\mathrm{rCER} = \mathrm{CER}(\mathrm{rom}(\boldsymbol{r}), \mathrm{rom}(\boldsymbol{h})), \qquad \mathrm{sWER} = \mathrm{WER}(\mathrm{skel}(\boldsymbol{r}), \mathrm{skel}(\boldsymbol{h})).$$

**Worked example 13.7: Two spellings of the same words**

कंप्यूटर साइंस $\xrightarrow{\text{rom}}$ "kampyutar sains", while "computer science" stays "computer science". Under rom the two still differ in several letters (partial credit in rCER), but their skeletons are *identical*: both become "kmptr sns", so sWER counts no error. (Output of `python cs_metric.py`.)

These scores are loose (partial credit for phonetically close spellings) and are applied identically to every system, so they are suitable for *ranking* systems on code-switched speech, not as absolute accuracies.

**In the code**

`analysis.py` reads the per-utterance hypotheses (`reports/hyps/*.jsonl`, written by every evaluation script with `--hyps`) and produces `reports/analysis.md`: bootstrap CIs, paired tests vs Stage 2, rCER/sWER on MUCS, the ML-SUPERB summary and the $7 \times 7$ LID confusion matrices. It regenerates byte-identically from the stored hypotheses.

Chapter 14

# Results

All numbers: held-out test split, 100 utterances per set, the same utterances for every system (Chapter 13). Sources: `reports/experiments.md`, `reports/analysis.md` and `reports/baselines/summary.md`.

## 14.1 Speech recognition: the full table

"Oracle" = true language token forced; "pred." = the model's own LID decides. "+fixes" = the two decode-time fixes of Chapter 15 (they change only English decoding and the LID decision).

| | Baselines (CER) | | Stage 2 (CER) | | | |
|---|---|---|---|---|---|---|
| Set | vanilla turbo | IndicConformer | oracle | pred. | +fixes, pred. | token-LID |
| FLEURS as | 1.204 | 0.063 | 0.084 | 0.085 | 0.084 | 0.98 |
| FLEURS bn | 0.482 | 0.078 | 0.094 | 0.094 | 0.094 | 1.00 |
| FLEURS hi | 0.127 | 0.065 | 0.077 | 0.077 | 0.077 | 1.00 |
| FLEURS mr | 0.251 | 0.063 | 0.075 | 0.080 | 0.080 | 0.99 |
| FLEURS or | 1.023 | 0.065 | 0.145 | 0.145 | 0.145 | 1.00 |
| FLEURS te | 0.489 | 0.087 | 0.086 | 0.086 | 0.086 | 1.00 |
| FLEURS en | 0.036 | – | 0.042 | 0.042 | 0.042 | 1.00 |
| IndicVoices as | 2.601 | 0.055 | 0.164 | 0.224 | 0.186 | 0.66 |
| IndicVoices bn | 1.185 | 0.061 | 0.119 | 0.159 | 0.143 | 0.77 |
| IndicVoices hi | 0.321 | 0.068 | 0.107 | 0.119 | 0.120 | 0.81 |
| IndicVoices mr | 0.716 | 0.060 | 0.124 | 0.140 | 0.140 | 0.85 |
| IndicVoices or | 2.046 | 0.104 | 0.196 | 0.223 | 0.223 | 0.75 |
| IndicVoices te | 1.116 | 0.081 | 0.203 | 0.209 | 0.209 | 0.95 |
| Kathbath bn | 0.505 | 0.021 | 0.044 | 0.052 | 0.052 | 0.99 |
| Kathbath hi | 0.116 | 0.029 | 0.055 | 0.061 | 0.061 | 0.99 |
| Kathbath mr | 0.240 | 0.029 | 0.065 | 0.066 | 0.066 | 0.99 |
| Kathbath or | 1.299 | 0.026 | 0.041 | 0.041 | 0.041 | 1.00 |
| Kathbath te | 0.371 | 0.037 | 0.060 | 0.060 | 0.060 | 1.00 |
| Lahaja hi | 0.259 | 0.038 | 0.064 | 0.152 | 0.147 | 0.77 |
| LibriSpeech en | 0.014 | – | 0.008 | 0.008 | 0.008 | 1.00 |
| Svarah en | **0.032** | – | 0.227 | 0.737 | 0.311 | 0.13 |
| MUCS hi–en | 0.512 | 0.545 | **0.306** | 0.306 | 0.306 | 1.00 |
| MUCS bn–en | 1.053 | 0.501 | **0.213** | 0.213 | 0.213 | 1.00 |
| **Mean (23 sets)** | 0.696 | (0.104 over 20) | **0.113** | 0.147 | **0.126** | |

### What the table says

1. **Fine-tuning turns an unusable model into a usable one** on every Indic set: mean CER 0.696 → 0.113. Every vanilla CER above 1 was a loop.
2. **Odia works** through the new token (Kathbath 0.041, FLEURS 0.145). FLEURS Odia is the weakest FLEURS language; it has the least FLEURS training data (3.4 h).

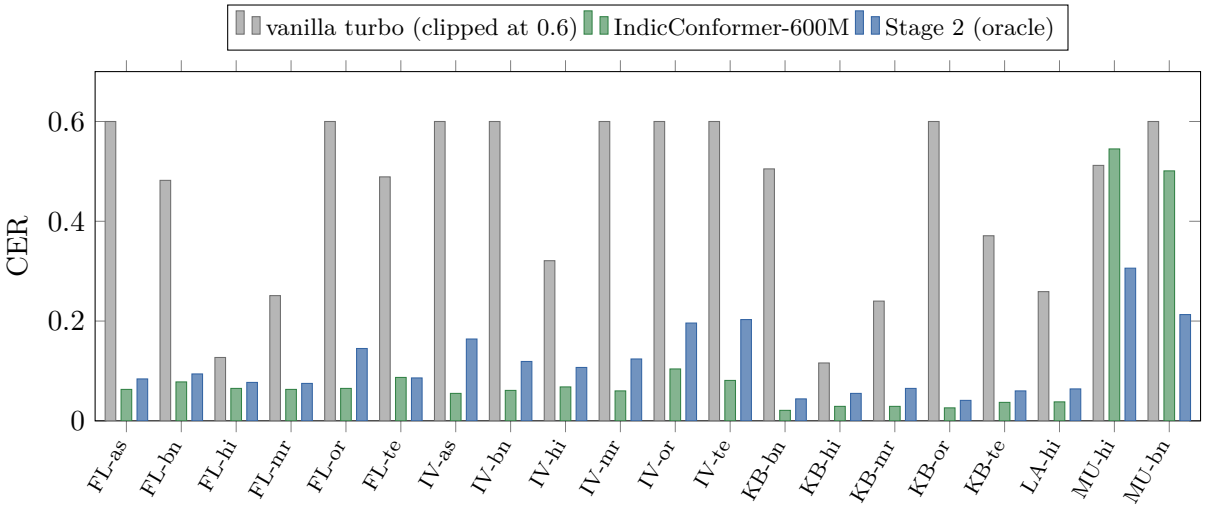



Figure 14.1: Indic and code-switched test sets. Grey bars at 0.6 are clipped; the true values are above 1 (repetition loops).

3. **English is preserved** on read speech (LibriSpeech 0.014 → 0.008; FLEURS 0.036 → 0.042) but **broken on Indian-accented English** (Svarah 0.032 → 0.227). Chapter 15 is devoted to this.
4. **IndicConformer is better on monolingual in-domain sets.** The paired bootstrap interval excludes 0 on 17 of the 18 monolingual Indic sets (all except FLEURS Telugu; FLEURS Hindi only just). The gap is largest on IndicVoices (0.04–0.12). Two caveats: IndicConformer was trained on far more AI4Bharat data from the *same sources* as IndicVoices and Kathbath, and it has no English, no LID and no code-switch ability. On FLEURS, which is out of domain for both, the gap is small (0.01–0.02 except Odia).
5. **Code-switching: Stage 2 is clearly best** (MUCS hi 0.306 vs 0.545, bn 0.213 vs 0.501), significant in the paired test.

## 14.2 The cost of LID errors

Mean CER: oracle 0.113, predicted 0.147, predicted with the calibrated rule 0.126. Without Svarah the routing cost is small almost everywhere; it is large exactly where LID is weak:

| Set | oracle | predicted (+fixes) | gap |
|---|---|---|---|
| Lahaja hi | 0.064 | 0.147 | +0.083 |
| IndicVoices as | 0.164 | 0.186 | +0.022 |
| IndicVoices bn | 0.119 | 0.143 | +0.024 |
| Svarah en | 0.227 (0.114 ASCII) | 0.311 | +0.197 |

## 14.3 Language identification

Pooled recall over all test sets (rows: true language):

| Language | vanilla token | ECAPA (closed set) | Stage 2 token | Stage 2 head | Stage 2 calibrated |
|---|---|---|---|---|---|
| hi | 0.84 | 0.90 | 0.91 | 0.91 | 0.91 |
| en | 1.00 | 0.71 | 0.71 | 0.75 | **0.84** |
| te | 0.14 | 0.97 | 0.98 | 0.98 | 0.98 |
| as | 0.00 | 0.92 | 0.82 | 0.92 | 0.89 |
| or | no token | no class | 0.92 | 0.90 | 0.92 |
| bn | 0.42 | 0.81 | 0.94 | 0.94 | 0.94 |
| mr | 0.52 | 0.89 | 0.94 | 0.93 | 0.94 |

Confusion matrix of the Stage 2 decoder token (counts; rows true, columns predicted):

| | hi | en | te | as | or | bn | mr | recall |
|---|---|---|---|---|---|---|---|---|
| hi | **457** | · | 7 | 1 | 17 | 3 | 15 | 0.91 |
| en | 26 | **213** | 21 | 1 | 15 | 16 | 8 | 0.71 |
| te | · | · | **295** | · | 4 | · | 1 | 0.98 |
| as | 3 | · | 4 | **164** | 22 | 7 | · | 0.82 |
| or | 9 | · | 8 | · | **275** | 1 | 7 | 0.92 |
| bn | 5 | · | 5 | · | 11 | **376** | 3 | 0.94 |
| mr | 3 | · | 3 | · | 5 | 6 | **283** | 0.94 |

- FLEURS, Kathbath, LibriSpeech and MUCS sets are at 0.98–1.00. Errors concentrate in spontaneous IndicVoices (0.66–0.95), Lahaja (0.77) and Svarah.
- **Excluding Svarah, English recall is 1.00** (200/200). All 87 English errors are Indian-accented English, spread over every Indic language.
- The `<|or|>` column absorbs errors (17 hi, 22 as, 15 en, 11 bn), as discussed in Chapter 9.
- The LID head helps most on Assamese (0.82 → 0.92); the calibrated rule combines both opinions plus an English bias.
- Against the dedicated ECAPA LID model, Stage 2 is equal or better on every Indic language and also covers Odia. ECAPA fails on Indian English in exactly the same way (Svarah 0.14), which suggests the difficulty is intrinsic to accented English, not to our model alone.

## 14.4 Robustness

**Hallucination** (Stage 2, 100 held-out noise/music clips per language): 0.01, 0.00, 0.01, 0.01, 0.01, 0.01, 0.01 for hi, en, te, as, or, bn, mr; mean output tokens on noise 0–1.5; false silence on real speech 0.00 in all languages. Vanilla: about 1.0.

## 14.5 Code-switching with the tolerant metrics

| System | set | CER | rCER | sWER |
|---|---|---|---|---|
| Stage 2 | MUCS hi | **0.306** | **0.251** | **0.294** |
| IndicConformer | MUCS hi | 0.545 | 0.367 | 0.387 |
| vanilla turbo | MUCS hi | 0.512 | 0.366 | 0.409 |
| Qwen3-ASR-1.7B | MUCS hi | 0.482 | 0.306 | 0.316 |
| Qwen3-ASR-0.6B | MUCS hi | 0.417 | 0.307 | 0.356 |
| Stage 2 | MUCS bn | **0.213** | **0.173** | **0.254** |
| IndicConformer | MUCS bn | 0.501 | 0.326 | 0.439 |
| vanilla turbo | MUCS bn | 1.053 | 0.749 | 1.382 |

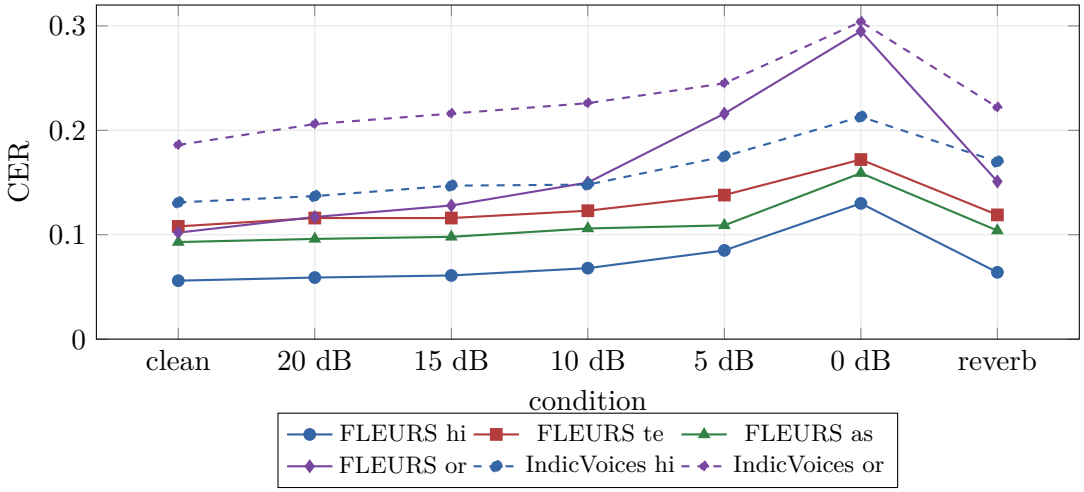



Figure 14.2: Stage 2 under held-out noise (60 utterances per set, `reports/snr_s2_turbo.json`). Degradation is gradual; Odia is the most noise-sensitive. Compare vanilla Whisper-small on FLEURS hi: 0.357 clean, 0.806 at 0 dB.

The ranking survives the script-tolerant metrics, so Stage 2's advantage is not an artefact of choosing the same script as the references.

## 14.6 Against Qwen3-ASR on Hindi and English

| Set | Qwen3-0.6B | Qwen3-1.7B | Stage 2 oracle | Stage 2 +fixes pred. |
|---|---|---|---|---|
| FLEURS hi | 0.107 | 0.068 | 0.077 | 0.077 |
| IndicVoices hi | 0.144 | 0.100 | 0.107 | 0.120 |
| Kathbath hi | 0.077 | **0.044** | 0.055 | 0.061 |
| Lahaja hi | 0.118 | 0.076 | 0.064 | 0.147 |
| MUCS hi–en | 0.417 | 0.482 | **0.306** | 0.306 |
| FLEURS en | 0.036 | **0.027** | 0.042 | 0.042 |
| LibriSpeech en | 0.008 | 0.006 | 0.008 | 0.008 |
| Svarah en | 0.134 | **0.101** | 0.227 (0.114 ASCII) | 0.311 |

Paired tests vs Stage 2 (oracle): Qwen3-1.7B is significantly better on Kathbath hi, FLEURS en and Svarah; not significantly different on FLEURS hi, IndicVoices hi and Lahaja; significantly worse on MUCS hi. Stage 2 is significantly better than Qwen3-0.6B on all Hindi sets. A 1.7B model trained for Hindi and English is level with our 7-language model on monolingual Hindi and clearly behind on code-switching.

## 14.7 Summary in the ML-SUPERB style

| System | sets | mean CER | worst set | std |
|---|---|---|---|---|
| Stage 2 (oracle) | 23 | 0.113 | MUCS hi 0.306 | 0.072 |
| Stage 2 + fixes (pred.) | 23 | 0.126 | Svarah 0.311 | 0.081 |
| Stage 2 (pred., no fixes) | 23 | 0.147 | Svarah 0.737 | 0.145 |
| vanilla turbo | 23 | 0.696 | IndicVoices as 2.601 | 0.650 |
| IndicConformer | 20 | 0.104 | MUCS hi 0.545 | 0.142 |

**Viva tip: the one-sentence summary**

"One 809 M-parameter Whisper with 20 M trained LoRA parameters, a new Odia token and an encoder LID head transcribes seven languages at a mean CER of 0.113 (0.126 with its own LID), almost never hallucinates on noise, beats every baseline on code-switched speech, trails the in-domain Indic specialist on monolingual Indic speech, and regressed on Indian-accented English."

Chapter 15

# The Indian-English Regression and Two Decode-Time Fixes

## 15.1 The symptom

Svarah is Indian-accented English (117 speakers from 65 districts). Vanilla turbo transcribes it at CER 0.032 and identifies it as English 99% of the time. After fine-tuning:

- token LID 0.13 (head 0.25): Svarah is now mostly called Hindi, Telugu, Bengali, Odia…;
- even with `<|en|>` forced (oracle), CER 0.227; with predicted LID, 0.737.

## 15.2 Diagnosis from the per-utterance hypotheses

| Subset of the 100 Svarah test utterances (oracle `<\|en\|>`) | $n$ | Stage 2 CER | vanilla CER |
|---|---|---|---|
| hypothesis contains non-Latin script | 39 | 0.715 | 0.059 |
| reference contains digits | 12 | 0.585 | 0.040 |
| rest | 55 | 0.048 | 0.023 |

(6 utterances are in both of the first two rows.)

Two mechanisms:

1. **Script leakage.** In 39% of utterances, even when told "English", the model writes English words in Bengali, Telugu or Devanagari script. The training mix contains no Indian-accented English: the only English is LibriSpeech (American/British audiobooks) and FLEURS. Indian-accented English is acoustically closer to the Indic training data (and to the English words inside MUCS, which are often written in Indic script) than to LibriSpeech, so the model maps it to Indic outputs.
2. **Number style.** References use digits ("25"); the model, like LibriSpeech, spells numbers out ("twenty five"). The normaliser does not verbalise digits (Chapter 3), so this counts as errors.

**Intuition: catastrophic forgetting, in its classic form**

Vanilla Whisper handled Indian English well because its 680,000-hour pretraining included it. Our fine-tuning distribution contained none, and nothing in the objective rewarded keeping that ability. Fine-tuning optimises the training mixture; abilities outside it are free to drift.

## 15.3 Fix 1: ASCII-only English decoding

**Definition 15.1: Logit masking**

Let $\mathcal{A} \subset \{1..V\}$ be the tokens whose decoded string is pure ASCII. When decoding with `<|en|>`, set $z_{t,v} = -\infty$ for $v \notin \mathcal{A}$ at every step. Then

$$\tilde{p}(v \mid \cdot) = \frac{e^{z_v} \mathbb{1}[v \in \mathcal{A}]}{\sum_{u \in \mathcal{A}} e^{z_u}} = p(v \mid \cdot,\ v \in \mathcal{A}),$$

the model's distribution *conditioned* on producing an ASCII token.

11,707 vocabulary tokens are not pure ASCII. The effect is surgical: Svarah oracle CER 0.227 → **0.114**, and no other English or MUCS set changes (MUCS is decoded with `<|hi|>`/`<|bn|>`, not `<|en|>`).

## 15.4 Fix 2: a calibrated LID rule

Fix 1 only helps if `<|en|>` is chosen. The second fix improves the decision itself by combining three pieces of evidence.

**Definition 15.2: Calibrated LID score**

For each $\ell \in \mathcal{L}$,

$$\text{score}(\ell) = \log \hat{p}_{\text{tok}}(\ell \mid \boldsymbol{x}) + b \cdot \mathbb{1}[\ell = \text{en}] + w \cdot \log q(\ell \mid \boldsymbol{x}), \qquad \hat{\ell} = \arg\max_{\ell} \text{score}(\ell),$$

where $\hat{p}_{\text{tok}}$ is the closed-set token probability (Definition 5.5) and $q$ the normalised head mass (Chapter 10, clamped at $10^{-6}$).

**Proposition 15.3: The rule is a weighted product of experts**

$\hat{\ell} = \arg\max_{\ell}\ \hat{p}_{\text{tok}}(\ell)\, q(\ell)^{w}\, e^{b\mathbb{1}[\ell=\text{en}]}$, i.e. the argmax of the normalised distribution $\pi(\ell) \propto \hat{p}_{\text{tok}}(\ell)\, q(\ell)^{w}\, e^{b\mathbb{1}[\ell=\text{en}]}$.

*Proof.* exp is increasing, so arg max of the score equals arg max of its exponential, which is the product. Normalising by a positive constant does not change the argmax. □

$w$ sets how much the encoder head is trusted relative to the decoder token; $b$ is a prior log-odds shift toward English.

**Fitting.** A grid $w \in \{0, 0.25, 0.5, 1, 2\}$, $b \in \{0, 0.25, \ldots, 30\}$ (605 points). Objective: *macro* LID accuracy over the calibration sets, ties broken toward the smallest change $(-|b| - w)$. Calibration data: the dev split (100 per set) plus a **speaker-disjoint Svarah calibration set**: Svarah has no speaker ids, so a speaker is approximated by the demographic profile (gender, age group, native state and district, qualification, occupation; 115 profiles). All profiles occurring in the 100 scored test utterances are excluded; 300 utterances from 54 remaining profiles are used. The test split is never used for fitting.

**Result.** $b = 22.25$, $w = 2.0$; calibration macro LID 0.900 → 0.919.

**Worked example 15.4: What $b = 22.25$ means**

In probability terms the English prior is multiplied by $e^{22.25} \approx 4.6 \times 10^9$. That sounds absurd, but the grid search kept gaining calibration accuracy as $b$ grew to this size. An utterance flips to English only if its score gap $\text{score}(\ell_{\text{win}}) - \text{score}(\text{en})$ without the bias is below $b$, so many Svarah utterances must have had gaps of the order of 20 nats: the fine-tuned token is extremely over-confident. Yet on genuine Indic speech the gap in favour of the true language is evidently even larger, because the same shift almost never flips a correct answer: on test no set loses more than 0.02 LID accuracy.

| Test set | LID before | LID after | pred. CER before | pred. CER after |
|---|---|---|---|---|
| Svarah en | 0.13 | 0.51 | 0.737 | 0.311 |
| IndicVoices as | 0.66 | 0.77 | 0.224 | 0.186 |
| IndicVoices bn | 0.77 | 0.78 | 0.159 | 0.143 |
| Lahaja hi | 0.77 | 0.78 | 0.152 | 0.147 |

Mean predicted-LID CER over 23 sets: 0.147 → 0.126.

**Viva tip: honest caveats about the calibration**

(1) $w = 2.0$ is the edge of the grid; but the macro gain from $w = 0.5$ to 2 is only 0.003, so extending the grid would change little. (2) The calibration Svarah utterances come from the Svarah corpus itself (different speakers, same recording conditions), so the fix is tuned to this kind of accent and microphone. (3) Speakers are approximated by profile; two people with the same profile would be merged (conservative for disjointness).

## 15.5 What would fix it properly

Svarah is still far behind vanilla (0.311 vs 0.032). The decode-time fixes treat the symptom. A real fix changes training:

- add Indian-accented English to the mix (Svarah-like data, NPTEL lecture audio);
- or a *rehearsal / distillation* term that keeps the fine-tuned model's English outputs close to the vanilla model's, e.g. add $\beta\,\mathrm{KL}\big(p_{\text{vanilla}}(\cdot \mid \boldsymbol{x}) \,\|\, p_\theta(\cdot \mid \boldsymbol{x})\big)$ on English audio;
- and verbalise digits consistently in references and targets.

Each needs a retraining run of about a day.

**In the code**

`eval_whisper.py --en_ascii --lid_rule runs/s2_turbo/lid_rule.json`; `make_svarah_calib.py` builds the calibration set; `lid_calib.py` fits $(b, w)$ and writes `lid_rule.json`. The demo uses both fixes.

**Check yourself 15.5: Why must the Svarah calibration utterances be speaker-disjoint from the scored ones?**

*Solution.* $b$ is fitted to maximise accuracy on the calibration set. If the scored test speakers were in it, $b$ could be tuned to those speakers' voices and the reported test improvement (0.13 → 0.51) would be optimistic: the same test-set contamination that speaker-disjoint splits prevent in training (Chapter 6).

Chapter 16

# The Demo: Long Audio, VAD Chunking and One Language Decision

## 16.1 What the demo does

`src/demo.py` is a Gradio web app: upload or record audio; it shows the detected language with closed-set confidences, the encoder head's opinion, and the transcript. The language can also be forced from a drop-down.

## 16.2 Why the audio must be cut

The model was trained on single utterances of at most 30 s and at most 440 text tokens. Real recordings are longer. The first version simply cut fixed 30-s windows. Two failures were observed:

1. On a packed 30-s window of a 51-s clip the model *skipped a whole sentence*, and the LID head's posteriors became nearly uniform (it had only seen single utterances).
2. 25-s Telugu chunks overflowed the 440-token budget (Telugu costs up to about 45 tokens per second; Exercise 2.8).

## 16.3 VAD-based chunking

**Definition 16.1: Voice activity detection (VAD)**

A VAD model labels short frames as speech or non-speech. Silero VAD (a small neural network) returns speech segments $[s_1, e_1], [s_2, e_2], \ldots$ in samples (minimum pause here: 200 ms).

**Definition 16.2: The chunking rule**

Let $L_{\max} = 12\,\text{s} \cdot 16000$. If the clip is at most $L_{\max}$, keep it whole. Otherwise the candidate cut points are the midpoints of the pauses, $c_k = \lfloor (e_k + s_{k+1})/2 \rfloor$. Starting at 0, repeatedly cut at the *last* candidate within $L_{\max}$ of the current start (greedy packing); if none exists (a single stretch of speech longer than 12 s), cut hard at $L_{\max}$. Chunks shorter than 0.3 s are dropped unless nothing else is left.

Cutting in pauses keeps words intact; greedy packing makes chunks as long as allowed, which gives the decoder more context. No audio is discarded.

## 16.4 One language for the whole clip

Each chunk $c$ gives a score vector $\text{score}_c(\ell)$ (token log-probability plus the calibrated rule terms of Chapter 15). The clip's language is

$$\hat{\ell} = \arg\max_{\ell} \sum_{c} \text{score}_c(\ell),$$

and the displayed confidence is $\text{softmax}_\ell\left(\sum_c \text{score}_c(\ell)\right)$.

**Proposition 16.3: Summing log-scores is naive-Bayes pooling**

If chunks were conditionally independent given the language and each $\text{score}_c$ were a log-likelihood, then $\sum_c \text{score}_c(\ell) = \log \prod_c p(\text{chunk}_c \mid \ell)$, the log-likelihood of the whole clip, and its argmax is the maximum-likelihood language.

*Proof.* Independence turns the joint likelihood into a product; log turns the product into a sum. □

In practice chunks are not independent (same speaker, same channel), so the pooled confidence is over-confident for long clips; the argmax is still a sensible decision.

All chunks are then decoded with the chosen language (ASCII-only if English), and the transcripts are joined with spaces.

## 16.5 Measured quality

On 50–70 s concatenations of FLEURS utterances the chunked demo gives CER 0.05 (Assamese), 0.15 (Hindi) and 0.18 (Telugu).

**Viva tip: bug fixed in the fourth review**

Gradio delivers integer PCM. The first version detected "integer audio" by checking $\max|x| > 1$ *after* casting to float: silent int16 audio was not rescaled, and int32 audio was divided by 32768 instead of $2^{31}$. It now checks the dtype and divides by `np.iinfo(dtype).max+1`.

Chapter 17

# Bugs, Lessons, Limitations and Future Work

## 17.1 The bugs that would have changed conclusions

The project log records 26 bugs. Most were cosmetic or caught before they mattered. These would have changed reported numbers or decisions:

| # | What was wrong | What it would have done |
|---|---|---|
| 4 | Official IndicVoices/Kathbath "valid" sets share 1,217 / 40 speakers with "train" | optimistic test numbers (test speakers seen in training) |
| 5 | Baseline decoding capped at 200 new tokens; Indic references reach 650 tokens | truncated outputs, wrong baseline CER |
| 9 | LID-CTC loss not normalised by frames (≈300 at start) | auxiliary loss would swamp the ASR loss |
| 10 | `begin_suppress_tokens` forbids EOT first, so empty output impossible | hallucination stuck near 100%; led to a wrong diagnosis and an unnecessary 30× loss weight |
| 11 | 92 "RIRs" were noise recordings; multi-mic averaging | reverb augmentation and reverb test partly noise |
| 12 | speed perturbation stretched clips past 30 s while labels stayed full | taught the model to hallucinate cut-off words |
| 13 | random language label on non-speech clips was a target | label noise in both LID outputs |
| 15 | `convert_tokens_to_ids("<\|or\|>")` returns EOT silently | Odia evaluation prompted with EOT |

## 17.2 Lessons

1. **Verify the measurement before the model.** A metric that does not move at all (bug 10) is more often a measurement bug than a modelling failure.
2. **Check official splits.** "Valid" does not mean "speaker-disjoint".
3. **Silent fallbacks are the dangerous ones.** An unknown token mapped to EOT, a cap of 200 tokens, a noise file in the RIR folder: none raised an error.
4. **Test a suspicion before acting on it.** The emb-lr sweep showed that the suspected cause of an all-Odia transient was wrong (Chapter 9).
5. **Make every random choice a function of (seed, epoch, index).** It made resumption exact and refactoring verifiable: after the final code cleanup, the test evaluation reproduced all 23 Stage 2 results and all IndicConformer results exactly.
6. **Fine-tuning optimises the mixture you give it.** Anything outside it (Indian English) may be forgotten.
7. **Unified memory is shared.** Three parallel Whisper-small runs filled 122 GB and all stalled at step 1; runs were made sequential.

## 17.3 Limitations

- $n = 100$ per test set and **one training seed**: bootstrap CIs cover test sampling, not training variance.
- One epoch of one deterministic mix; IndicVoices is sub-sampled to 0.5–0.7× per epoch.
- IndicConformer's advantage may be partly in-domain; FLEURS (out of domain for both) shows a smaller gap.
- MUCS references mix scripts inconsistently; rCER and sWER are loose comparison metrics, not absolute accuracies.
- Digits vs number words are not normalised; parenthesised FLEURS text is dropped (inflates FLEURS CER by 0.005–0.02 for all systems).
- The robustness test noise comes from the same MUSAN categories as the training noise (held-out files only); no UrbanSound or DEMAND test was run. MUSAN "music" includes some tracks with vocals, used as empty-target non-speech.
- Not done: synthetic Hinglish via TTS, the words-per-second truncation guard (loop rate is already about 0), Voice of India and Vistaar benchmarks (not available).

## 17.4 Future work

1. Indian-accented English in training, or a distillation term toward vanilla Whisper on English (Chapter 15).
2. A second epoch on a reseeded mix (`balanced_mix.py 1`); the dev curve had not flattened.
3. Test why `<|or|>` absorbs LID errors, e.g. by also unfreezing the other six language rows with the same mask.
4. Multiple seeds for the final configuration.
5. Fix the normaliser's parenthesis rule for FLEURS and verbalise numbers consistently.

Chapter 18

# Formula Sheet and Viva Practice

## 18.1 One-page formula sheet

**Audio and Whisper**

$x[n] = x_c(n/f_s)$, $f_s = 16$ kHz · STFT 25 ms window, 10 ms hop · $\mathrm{mel}(f) = 2595 \log_{10}(1 + f/700)$ · $\boldsymbol{F} = (\max(L, \max L - 8) + 4)/4$, $L = \log_{10} \max(\mathcal{M}, 10^{-10})$ · input $128 \times 3000$ · encoder 1500 frames, 50/s · $T = \min(1500, \lfloor 50D \rfloor + 1)$ · decoder $\leq 448$ positions; text $\leq 440$ tokens · Indic 17–45 tokens/s, English $\approx 3$.

**Metrics and statistics**

$D[i,j] = \min\{D[i-1,j]+1,\ D[i,j-1]+1,\ D[i-1,j-1]+\mathbb{1}[r_i \neq h_j]\}$ · $\mathrm{CER} = \sum(S+D+I)/\sum N$ (can exceed 1) · $|m-n| \leq d \leq \max(m,n)$ · bootstrap: resample utterances with replacement, $B = 2000$, percentile 95% CI · paired: same indices for both systems, significant if CI of $\Delta$ excludes 0 · routing cost $= \mathrm{CER}_{\mathrm{pred}} - \mathrm{CER}_{\mathrm{oracle}}$.

**Model and training**

$p(\boldsymbol{y} \mid \boldsymbol{x}) = \prod_t p(y_t \mid \boldsymbol{y}_{<t}, \boldsymbol{x})$ · $\nabla_{\boldsymbol{z}}(-\log \mathrm{softmax}(\boldsymbol{z})_y) = \boldsymbol{p} - \boldsymbol{e}_y$ · $\mathrm{Attn} = \mathrm{softmax}(QK^\top/\sqrt{d_k} + M)V$ · LoRA $W_0 + \frac{\alpha}{r}BA$, $B_0 = 0$, $r = 32$, $\alpha = 64$, params $r(d_{\mathrm{in}} + d_{\mathrm{out}})$ · trained LoRA 20.48 M (22 enc + 4 dec layers) · `<|or|>` = row 51866, init from `<|bn|>`, gradient mask, wd 0 · $\mathcal{L} = \mathcal{L}_{\mathrm{CE}} + 0.3\,\frac{1}{|\mathcal{S}|}\sum_i \ell_{\mathrm{CTC},i}/T_i$ · AdamW $(0.9, 0.98, 10^{-6})$, lr $3 \times 10^{-4}$ (LoRA), $2.5 \times 10^{-4}$ (row), $10^{-3}$ (head) · warm-up 10% then linear decay · clip norm 1 · $11{,}000 \times 32$.

**CTC, data, robustness, LID rule**

$p(\boldsymbol{y} \mid \boldsymbol{x}) = \sum_{\boldsymbol{\pi} \in \mathcal{B}^{-1}(\boldsymbol{y})} \prod_t p_t(\pi_t)$ · single label: $\sum_{s \leq e} \prod_{t<s} p_t(\varnothing) \prod_s^e p_t(\ell) \prod_{t>e} p_t(\varnothing)$, $T(T+1)/2$ paths · head decision $\arg\max_\ell \sum_t p_t(\ell)$ · mix: $s_c = h_c \min(3, h_{\max}/h_c)$, share cap 0.7, 90 h per language, total $7 \cdot 90/(1-0.1) = 700$ h · noise gain $g = \sqrt{P_s/(P_n 10^{\sigma/10})}$, $\sigma \sim \mathcal{U}(0,20)$ · speed $\pm 10\%$ only if $< 27$ s · 5% non-speech, target EOT, language masked · hallucination $=$ $> 5$ tokens on noise, with `begin_suppress_tokens` off · score $= \log \hat{p}_{\mathrm{tok}} + 22.25\,\mathbb{1}[\mathrm{en}] + 2 \log q$.

**Headline numbers (test, 23 sets × 100 utterances)**

Mean CER: vanilla 0.696 → Stage 2 oracle **0.113**, predicted 0.147, predicted + fixes **0.126** · IndicConformer 0.104 over its 20 sets (significantly better on 17 of 18 monolingual Indic sets) · MUCS hi/bn 0.306/0.213 vs IndicConformer 0.545/0.501 · hallucination $\approx 1.0 \rightarrow \leq 0.01$ · Svarah 0.032 (vanilla) vs 0.227 oracle, 0.114 ASCII, 0.311 predicted + fixes · Odia Kathbath 0.041, FLEURS 0.145 · 16.6 h training at 5.9 utt/s.

## 18.2 Likely viva questions, with answer sketches

**Q1. Why Whisper and not IndicConformer as the base?** IndicConformer has no English, no LID and no code-switch support; the project requires one model with integrated LID.

Whisper's decoder already has a language token and a multilingual text decoder, and turbo is fast. The price: worse in-domain monolingual Indic CER.

**Q2. Why does vanilla Whisper's CER exceed 1?** CER counts insertions; repetition loops make hypotheses much longer than references (Proposition 3.5).

**Q3. How did you add Odia?** New row 51866 in the tied embedding, initialised from Bengali, gradient-masked so only that row trains, own lr $2.5 \times 10^{-4}$, weight decay 0 (Proposition 9.4). Test LID 1.00 on FLEURS/Kathbath, CER 0.041 on Kathbath.

**Q4. Why initialise from Bengali?** Closest language and script among Whisper's tokens; the clone starts as an exact tie (Proposition 9.3) that training only has to break.

**Q5. What does the LID-CTC head add?** A frame-level LID signal straight into the encoder, and a second LID opinion. E2: token LID on IndicVoices Assamese 0.73 → 0.84; at scale the head lifts Assamese recall 0.82 → 0.92.

**Q6. Derive the single-label CTC probability.** Proposition 10.4: valid alignments are blanks, one run of $\ell$, blanks.

**Q7. Why divide the CTC loss by $T$?** Its scale grows linearly with $T$ ($\approx T \log 8$ at initialisation), which would dominate the token cross-entropy.

**Q8. Why LoRA and not full fine-tuning?** E1: LoRA equal or better (IndicVoices 0.108 vs 0.173), 18× fewer trainable parameters, merges into the base at no inference cost.

**Q9. How many parameters did you actually train?** 20.48 M LoRA + 1,280 (Odia row) + 10,248 (head); the logged 86.9 M includes the whole 66.4 M embedding matrix, of which only one row moves.

**Q10. Why balance the data, and how?** The mix weights define the objective (Proposition 7.1). Two-stage upsampling: within-language toward the largest corpus (cap 3×, share cap 70%), then 90 h per language. E3: Assamese 0.342 → 0.278 on IndicVoices.

**Q11. How do you know the test numbers are honest?** Speaker-disjoint splits (fixed 1,217 shared IndicVoices speakers), text-leakage removal, test used only for final models, paired bootstrap CIs, exact reproduction after refactoring.

**Q12. Why is a non-significant difference not "equal"?** The CI includes 0 but also includes differences of ±0.015; we cannot distinguish the systems with 100 utterances, which is different from showing they are the same.

**Q13. What went wrong with hallucination measurement?** `begin_suppress_tokens` forbids EOT at the first step, so an empty output was impossible (bug 10). With it disabled: 2–4% (Whisper-small), ≤1% (Stage 2).

**Q14. Why does Indian English fail and how did you mitigate it?** No Indian-accented English in training; accented English resembles Indic training audio, so it is mapped to Indic scripts and labels. ASCII-only English decoding (0.227 → 0.114 oracle) and a calibrated LID rule (LID 0.13 → 0.51). Real fix: training data or distillation.

**Q15. Is the calibrated rule fitted on test?** No: dev split plus a speaker-disjoint Svarah calibration set (profiles absent from the scored test utterances).

**Q16. Why not a denoiser?** Evidence that enhancement before strong ASR hurts; training-time augmentation instead. Stage 2 degrades gradually: FLEURS-hi 0.056 clean, 0.130 at 0 dB.

**Q17. What is the oracle-vs-predicted gap and what is yours?** The cost of LID errors: 0.113 → 0.126 mean with the calibrated rule; large where LID is weak (Lahaja 0.064 → 0.147).

**Q18. Why is the demo chunked at 12 s and not 30 s?** Packed 30-s windows skipped speech and confused the LID head; Indic token cost (up to ∼45 tokens/s) overflows the 440-token budget for long chunks.

**Q19. What would you do with one more day of GPU time?** Retrain with Indian-accented English (or a KL term to vanilla on English) on a reseeded second-epoch mix; that targets the largest remaining failure.

**Q20. Why is CER the primary metric?** Long agglutinative words, spelling variants and script mixing make WER overly harsh; CER gives partial credit at the character level.

Chapter A

# Reproduction and Code Map

## A.1 Code map

| File (src/) | Role |
|---|---|
| common.py | paths, language list, MAX_TEXT_TOKENS, special ids, enc_frames, noise clips, LID-head helpers |
| normalize.py, metrics.py | the single normaliser; WER/CER via jiwer |
| audit_data.py | metadata audit → reports/data_audit.csv |
| build_manifests.py | decode to 16 kHz FLAC, filters, JSONL manifests |
| make_splits.py | speaker-disjoint roles, leakage removal → splits/ |
| prep_tokens.py | Whisper token counts (ntok) |
| balanced_mix.py | two-stage balanced 700 h mix |
| augment.py | noise, RIR, speed, SpecAugment; 80/20 hash split of noise files |
| train_whisper.py | full/LoRA/QLoRA, Odia token, LID-CTC, non-speech, checkpoint/resume |
| load_ft.py | load vanilla/adapter/full model; correct Odia id |
| eval_whisper.py | oracle/predicted CER, LID, head LID, --en_ascii, --lid_rule, --hyps |
| eval_snr.py, eval_halluc.py | SNR/reverb sweep; hallucination and false silence |
| eval_indicconformer.py, ic_gpu.py | IndicConformer baseline (onnx2torch, greedy CTC) |
| eval_qwen3asr.py, eval_ecapa.py | Qwen3-ASR and ECAPA baselines (run in .venv-qwen) |
| make_svarah_calib.py, lid_calib.py | speaker-disjoint calibration set; fit $(b, w)$ |
| analysis.py, cs_metric.py | bootstrap, paired tests, rCER/sWER, confusion matrices |
| demo.py | Gradio demo with Silero-VAD chunking |
| summarize.py, baseline_table.py | regenerate reports/experiments.md, reports/baselines/summary.md |
| run_overnight.sh, run_stage2.sh | the exact command chains for E1–E4 and Stage 2 |

## A.2 Commands

From /home/shibam/IE643, source .venv/bin/activate, cd src:

```
python audit_data.py
python build_manifests.py                    # ~40 min
python make_splits.py && python prep_tokens.py && python balanced_mix.py 0
bash run_stage2.sh                           # Stage 2 (~16.6 h); re-running resumes
python make_svarah_calib.py && python lid_calib.py
python eval_whisper.py --model ../models/whisper-large-v3-turbo \
    --adapter ../runs/s2_turbo/adapter --split test --n 100 \
    --en_ascii --lid_rule ../runs/s2_turbo/lid_rule.json \
    --out ../reports/test_s2_turbo_fix.json --hyps ../reports/hyps/s2_turbo_fix.jsonl
python analysis.py                           # reports/analysis.md
python summarize.py                          # reports/experiments.md
python demo.py --port 7860
```

## A.3 Artefacts of the final model

- `runs/s2_turbo/adapter/`: LoRA weights plus the resized embedding containing the `<|or|>` row.
- `runs/s2_turbo/lid_head.pt`: the $8 \times 1280$ LID-CTC head and its class order.
- `runs/s2_turbo/lid_rule.json`: $b = 22.25$, $w = 2.0$ and the calibration results.
- `reports/hyps/*.jsonl`: one row per test utterance per system (reference, oracle and predicted hypotheses, LID).

## A.4 Things to know before re-using results

- E1–E4 are dev-set, single-seed, Whisper-small. Only Stage 2 (and the baselines) were evaluated on the test split.
- Use `LANGID` from `load_ft.load`; never look up the Odia id with the tokenizer's `convert_tokens_to_ids`.
- Measure hallucination only with `eval_halluc.py`; ignore `hallucination_rate` in old `snr_*.json`.
- Never run two Whisper trainings at once on this machine.